



CAPACITAÇÃO INTERNA · APOSTILA-GUIA

GIT, GITLAB E IA APLICADA À ENGENHARIA DE SOFTWARE

Do primeiro commit à documentação de sistemas legados com um agente de IA: um guia para quem está começando

Ronivaldo D. Andrade

Estagiário de Análise e Desenvolvimento de Sistemas
Marinha do Brasil

Capacitação em 01/10/2026, das 10h às 11h

GIT · IA · DOCS

SOBRE ESTA APOSTILA

Esta apostila acompanha a capacitação “**Git, GitLab e IA aplicada à Engenharia de Software**”, apresentada na Marinha do Brasil. A apresentação tem uma hora e é só expositiva. Aqui está tudo o que foi dito com mais calma, os detalhes que não cabem num slide e os **exercícios práticos**, para você aprender no seu ritmo.

Ela foi escrita para quem **nunca usou Git**. Se você já usa, pule direto para os capítulos 7 e 8, que contam os dois problemas reais resolvidos no estágio.

Capacitação	Git, GitLab e IA aplicada à Engenharia de Software
Autor e instrutor	Ronivaldo D. Andrade
Função	Estagiário de Análise e Desenvolvimento de Sistemas
Organização	Marinha do Brasil
Data da capacitação	01/10/2026, das 10h às 11h
Data de elaboração	28/09/2026
Material complementar	Apresentação de slides (apresentacao/apresentacao.pdf)

COMO LER ESTA APOSTILA

Ao longo do texto aparecem caixas coloridas. Cada cor tem um papel:

Conceito

Uma definição importante, para guardar.

Analogia

Uma comparação com o dia a dia, para a ideia ficar concreta.

Dica

Um atalho ou uma boa prática.

⚠ Atenção

Um erro comum ou um cuidado que evita dor de cabeça.

>_ TERMINAL

```
# Isto é um comando para digitar no terminal.  
# As linhas que começam com # são comentários: não precisa digitar.  
git status
```

🖨 SAÍDA

Isto é o que o computador responde. O texto exato pode variar conforme a versão e o idioma do seu Git.

Quando um comando traz algo entre <sinais de menor e maior>, como <URL_DO_REPOSITÓRIO>, troque tudo, inclusive os sinais, pelo valor real.

⚠ Antes de usar IA com código da instituição

O capítulo 8 mostra como um agente de IA ajudou a documentar sistemas. Um agente desse tipo envia trechos do código para um serviço externo. **Só use com sistemas institucionais quando houver autorização e respeitando as normas de segurança da informação da sua OM.** Para praticar, use projetos pessoais ou de código aberto.

SUMÁRIO

1	Antes de começar	1
1.1	Por que versionar?	1
1.2	O terminal, sem medo	1
1.3	Instalando o Git	2
1.4	Dizendo ao Git quem é você	2
2	Git: o controle de versão	4
2.1	Repositório e commit	4
2.2	As três áreas do Git	4
2.3	Os primeiros comandos	5
2.4	Branches	6
2.5	Merge: juntando linhas de trabalho	7
2.6	Conflitos	7
2.7	Trabalhando com um repositório remoto	9
2.8	Desfazendo coisas	9
2.9	Um aviso honesto	10
3	GitLab: colaboração em volta do Git	11
3.1	Git, GitLab, GitHub e glab	11
3.2	GitLab Self-Managed	12
3.3	Branch protegida	12
3.4	Merge Request	13
3.5	O fluxo básico de trabalho em equipe	14
4	GitLab CLI: o GitLab pelo terminal	15
4.1	Instalação	15
4.2	Autenticação	15

4.3	Os comandos mais úteis	16
4.4	Terminal ou interface web?	16
5	Padrões de commit	17
5.1	Por que padronizar	17
5.2	Conventional Commits	17
5.3	Boas práticas para a mensagem	19
5.4	Gitmoji: emojis nas mensagens	20
5.5	Um padrão combinando os dois	21
5.6	Ferramentas	21
6	Assinatura de commits com GPG e SSH	23
6.1	O problema: o autor de um commit é só um texto	23
6.2	GPG ou SSH?	24
6.3	Assinando com GPG	24
6.4	Assinando com SSH	27
6.5	Conferindo no GitHub e no GitLab	29
6.6	Cuidados com a chave	30
6.7	Outras plataformas e ferramentas	30
6.8	Problemas comuns	31
7	Problema 1: integrar um sistema já versionado ao GitLab institucional	32
7.1	O cenário	32
7.2	Por que não funciona do jeito óbvio	32
7.3	O objetivo	33
7.4	A solução, passo a passo	33
7.5	Resumo em oito linhas	37
7.6	Erros que podem aparecer	38
8	Problema 2: documentar sistemas legados com um agente de IA	39
8.1	Versionado não é documentado	39
8.2	A abordagem: IA como ferramenta de engenharia	39
8.3	A IA não substitui a validação humana	40
8.4	A evolução real dos prompts	40
8.5	O que deu errado e o que cada falha ensinou	42

8.6	A melhora, em números	43
8.7	O prompt final por dentro	44
8.8	Prompt em inglês, documentação em português	47
8.9	O que o prompt pede e o que ele entrega	47
8.10	Como usar na prática	48
9	Exercícios práticos	50
9.1	Parte 1 — Git no seu computador	50
9.2	Parte 2 — GitLab	52
9.3	Parte 3 — IA	53
9.4	Parte 4 — Desafio: reproduzir o Problema 1	55
9.5	Parte 5 — Padrões de commit	56
9.6	Parte 6 — Assinatura de commits	57
10	Para levar daqui	59
10.1	Os dois problemas, lado a lado	59
10.2	Conceitos fundamentais	59
10.3	Fluxos recomendados	60
10.4	Mensagem final	60
A	Cola de comandos	61
B	Glossário	63
C	Plano da apresentação	65
D	Créditos e licenças	66
E	Para continuar estudando	68

1

ANTES DE COMEÇAR

1.1 POR QUE VERSIONAR?

Quem já trabalhou num documento importante conhece esta pasta:

 relatorio.docx
 relatorio_v2.docx
 relatorio_v2_final.docx
 relatorio_v2_final_AGORA_VAI.docx
 relatorio_v2_final_corrigido(1).docx

Ninguém sabe qual é a versão certa, quem mudou o quê, nem por quê. Agora imagine isso com um sistema de milhares de arquivos e várias pessoas mexendo ao mesmo tempo. É esse o problema que um **sistema de controle de versão** resolve.

Com controle de versão, cada alteração fica registrada com:

- **o que** mudou, linha a linha;
- **quem** mudou e **quando**;
- **por que** mudou, numa mensagem escrita por quem fez;
- a possibilidade de **voltar** a qualquer estado anterior;
- várias pessoas trabalhando **em paralelo** sem apagar o trabalho umas das outras.

Analogia

Pense no Git como o “histórico de versões” de um documento compartilhado, só que muito mais poderoso: ele guarda o projeto inteiro, funciona sem internet e deixa cada pessoa trabalhar numa cópia própria antes de juntar tudo.

1.2 O TERMINAL, SEM MEDO

Quase tudo nesta apostila é feito no **terminal** (também chamado de linha de comando, console ou *shell*). É uma janela onde você digita uma ordem e aperta `Enter`.

- **Linux:** procure por “Terminal” no menu de aplicativos.
- **Windows:** depois de instalar o Git, use o **Git Bash**, que vem junto. Ele aceita os mesmos comandos mostrados aqui.

Cinco comandos básicos ajudam a se localizar:

<code>pwd</code>	mostra em que pasta você está
<code>ls</code>	lista os arquivos da pasta atual
<code>cd pasta</code>	entra numa pasta (<code>cd ..</code> volta uma pasta)
<code>mkdir nome</code>	cria uma pasta
<code>cat arquivo</code>	mostra o conteúdo de um arquivo

1.3 INSTALANDO O GIT

>_ TERMINAL · LINUX (DEBIAN/UBUNTU)

```
sudo apt update
sudo apt install git
```

No **Windows**, baixe o instalador oficial em <https://git-scm.com/downloads> e aceite as opções padrão. Ao final, abra o **Git Bash**.

Para conferir se deu certo:

>_ TERMINAL

```
git --version
```

SAÍDA

```
git version 2.34.1
```

Qualquer versão a partir da 2.28 funciona com todos os exemplos desta apostila. Se a sua for mais antiga, veja a dica da seção 2.3.

1.4 DIZENDO AO GIT QUEM É VOCÊ

Todo registro no Git leva o nome e o e-mail de quem o fez. Configure uma única vez por computador:

>_ TERMINAL

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
# faz a branch inicial de novos repositórios se chamar main
git config --global init.defaultBranch main
```

 Dica

Use o mesmo e-mail cadastrado no GitLab da instituição. Assim o GitLab reconhece os seus commits e mostra a sua foto e o seu nome ao lado deles.

Para ver o que está configurado:

>_ TERMINAL

```
git config --list
```

2

GIT: O CONTROLE DE VERSÃO

Git

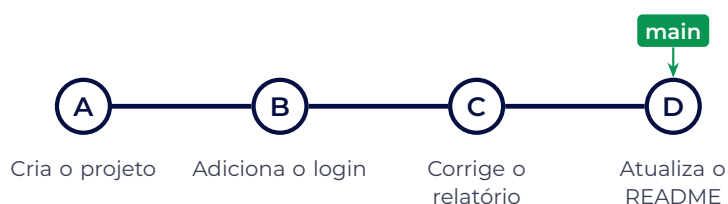
O Git é um **sistema distribuído de controle de versão**. Ele registra a evolução de um projeto ao longo do tempo. “Distribuído” quer dizer que cada pessoa tem uma cópia completa do histórico no próprio computador: dá para trabalhar sem servidor e sem internet.



2.1 REPOSITÓRIO E COMMIT

Um **repositório** é uma pasta cujo histórico o Git está acompanhando. O Git guarda esse histórico numa subpasta escondida chamada `.git`. Não mexa nela à mão.

Um **commit** é uma “fotografia” do projeto num momento, acompanhada de quem a tirou, quando e por quê. Os commits se encadeiam e formam o **histórico**:



Cada commit recebe um identificador único, um código como 4203776. Você vai ver esses códigos em todo lugar.

2.2 AS TRÊS ÁREAS DO GIT

Esta é a ideia que mais ajuda a entender os comandos. Um arquivo passa por três lugares:



🗨️ Analogia

É como preparar uma remessa. Você trabalha nos documentos na sua mesa (diretório de trabalho). Separa numa caixa os que vão ser enviados (área de preparação, com `git add`). Lacra a caixa e registra o envio no livro de protocolo (repositório, com `git commit`). O livro guarda cada remessa para sempre.

2.3 OS PRIMEIROS COMANDOS

>_ TERMINAL

```
mkdir meu-projeto
cd meu-projeto
git init -b main           # transforma a pasta num repositório
```

💡 Git anterior à versão 2.28

Se o seu Git não aceitar `-b main`, use `git init` e, logo depois do primeiro commit, `git branch -m main`.

Crie um arquivo e pergunte ao Git o que ele vê:

>_ TERMINAL

```
echo "Olá, Git!" > saudacao.txt
git status
```

🖥️ SAÍDA

```
No ramo main

No commits yet

Arquivos não monitorados:
  (utilize "git add <arquivo>..." para incluir o que será submetido)
  saudacao.txt
```

O Git viu o arquivo, mas ainda não o acompanha. Prepare e registre:

>_ TERMINAL

```
git add saudacao.txt
git commit -m "Cria a saudação"
```

SAÍDA

```
[main (root-commit) 4203776] Cria a saudação
1 file changed, 1 insertion(+)
create mode 100644 saudacao.txt
```

💡 Boas mensagens de commit

Escreva o **porquê** da mudança, de um jeito que outra pessoa entenda daqui a um ano. “Corrige o cálculo do total quando não há itens” é útil. “ajustes” não é.

Para consultar o histórico:

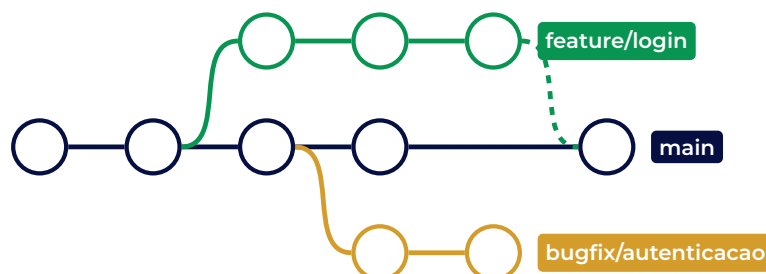
>_ TERMINAL

```
git log --oneline          # uma linha por commit
git log --oneline --graph --all # desenha as branches
git diff                  # o que mudou e ainda não foi preparado
```

2.4 BRANCHES

📖 Branch

Uma **branch** (ramo) é uma **linha de desenvolvimento** dentro do histórico. Ela permite trabalhar numa mudança de forma isolada, sem mexer na linha principal, a main.



Branches servem para desenvolver funcionalidades isoladamente, corrigir bugs, experimentar, trabalhar em paralelo, evitar alterações diretas na main e permitir revisão antes da integração. É comum usar prefixos que dizem o tipo do trabalho: feature/, bugfix/, hotfix/.

⚠ Branch não é cópia

Branch **não** é uma cópia do projeto. É só um **ponteiro** que aponta para um commit e anda para a frente a cada novo commit. Por isso criar uma branch é instantâneo, mesmo num projeto enorme.

>_ TERMINAL

```
git branch                # lista as branches (* marca a atual)
git switch -c feature/despedita  # cria e já entra na nova branch
git switch main            # volta para a main
```

2.5 MERGE: JUNTANDO LINHAS DE TRABALHO

O **merge** traz as mudanças de uma branch para a branch em que você está.

>_ TERMINAL

```
git switch main
git merge feature/despedita
```

SAÍDA

```
Updating 4203776..afe16bb
Fast-forward
 saudacao.txt | 1 +
 1 file changed, 1 insertion(+)
```

“Fast-forward” quer dizer que a main não tinha nada novo, então o Git só avançou o ponteiro. Quando as duas branches andaram, o Git cria um **commit de merge**, que tem dois “pais” e une as duas linhas.

2.6 CONFLITOS

📖 Conflito

Um conflito acontece quando duas branches mudaram **a mesma região do mesmo arquivo** de formas diferentes, e o Git não tem como saber qual versão vale. Conflito **não é erro**: é o Git pedindo uma decisão humana.

Exemplo: na main, a primeira linha virou “E aí, pessoal!”. Na feature/formal, virou “Prezados, bom dia!”. Ao juntar:

>_ TERMINAL

```
git merge feature/formal
```

SAÍDA

```
Mesclagem automática de saudacao.txt
CONFLITO (conteúdo): conflito de mesclagem em saudacao.txt
Automatic merge failed; fix conflicts and then commit the result.
```

O Git escreve as duas versões dentro do arquivo, separadas por marcadores:

SAÍDA · CONTEÚDO DE saudacao.txt

```
<<<<<< HEAD
E aí, pessoal!
=====
Prezados, bom dia!
>>>>>> feature/formal
Até logo!
```

<<<<<< HEAD	início da versão da branch em que você está
=====	separa as duas versões
>>>>>> feature/formal	fim da versão que está chegando

Para resolver:

1. abra o arquivo e deixe só o conteúdo que deve ficar (pode ser uma das versões ou uma combinação das duas);
2. apague as três linhas de marcadores;
3. marque como resolvido e registre:

>_ TERMINAL

```
git add saudacao.txt
git commit -m "Resolve o conflito da saudação"
```

💡 Dica

Se você se perdeu no meio de um merge, `git merge --abort` desfaz tudo e volta ao estado de antes do merge. Nada se perde.

2.7 TRABALHANDO COM UM REPOSITÓRIO REMOTO

Até aqui tudo foi local. Para trabalhar em equipe, o repositório também fica num servidor, como o GitLab. Esse servidor é chamado de **remoto**, e por convenção o principal se chama **origin**.

<code>git clone <URL></code>	baixa um repositório remoto inteiro, já configurado
<code>git remote -v</code>	mostra os remotos configurados
<code>git remote add origin <URL></code>	liga um repositório local a um remoto
<code>git fetch</code>	baixa as novidades do remoto sem mexer nos seus arquivos
<code>git pull</code>	baixa e já faz o merge na branch atual
<code>git push origin <branch></code>	envia os seus commits para o remoto

fetch × pull

`git fetch` é olhar a caixa de correspondência e trazer as cartas para a mesa, sem abrir. `git pull` é trazer e já arquivar tudo na pasta. O `fetch` é mais seguro quando você quer ver antes o que chegou.

2.8 DESFAZENDO COISAS

<code>git restore arquivo</code>	descarta as mudanças não preparadas do arquivo
<code>git restore --staged arquivo</code>	tira o arquivo da área de preparação, mantendo as mudanças
<code>git revert <commit></code>	cria um novo commit que desfaz outro (seguro em branch compartilhada)
<code>git merge --abort</code>	cancela um merge em andamento

Atenção

`git restore arquivo` apaga as mudanças que ainda não viraram commit. Não há como recuperá-las depois. Confira com `git diff` antes.

2.9 UM AVISO HONESTO



“Isto é o Git. Ele controla trabalho colaborativo em projetos por meio de um belo modelo distribuído de árvore da teoria dos grafos.”

“Legal. Como a gente usa?”

“Não faço ideia. Só decora estes comandos de terminal e digita para sincronizar. Se der erro, salva teu trabalho em outro lugar, apaga o projeto e baixa uma cópia nova.”

xkcd #1597, “Git”, de Randall Munroe (<https://xkcd.com/1597/>), licença CC BY-NC 2.5. Tradução livre.

A tirinha é engraçada porque é verdade para muita gente. O objetivo deste capítulo foi o contrário: entender o **modelo** (commits, ponteiros, três áreas), para que os comandos façam sentido e você nunca precise apagar o projeto.

3

GITLAB: COLABORAÇÃO EM VOLTA DO GIT

GitLab

O GitLab é uma **plataforma de desenvolvimento e gerenciamento de software baseada no Git**. Ele hospeda repositórios e acrescenta, num só lugar, o que uma equipe precisa para colaborar: revisão de código, controle de permissões, Merge Requests, Issues, CI/CD, pipelines, releases, segurança e auditoria.

3.1 GIT, GITLAB, GITHUB E GLAB

Esses nomes confundem porque são parecidos. Mas **Git** e **GitLab** não são a mesma coisa:

	O que é	Principal finalidade
 Git	Sistema distribuído de controle de versão	Controlar o histórico do código
 GitLab	Plataforma baseada em Git, com opção de instalação na própria infraestrutura	Hospedar, gerenciar e colaborar sobre projetos
 GitHub	Plataforma baseada em Git, oferecida principalmente como serviço em nuvem	Hospedar, gerenciar e colaborar sobre projetos
 GitLab CLI (glab)	Ferramenta de linha de comando	Interagir com o GitLab pelo terminal

Analogia

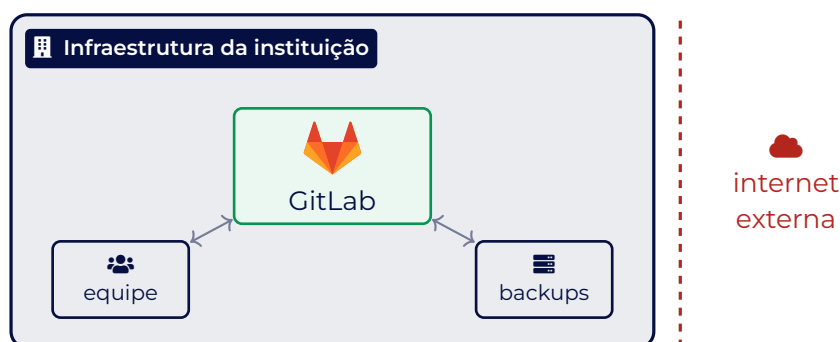
O **Git** é o motor do carro. O **GitLab** e o **GitHub** são garagens com oficina, vistoria e controle de quem entra. O **glab** é o controle remoto do portão da garagem do GitLab.

O Git funciona sozinho, sem GitLab nem GitHub. Os comandos `git init`, `git add` e `git commit` do capítulo anterior não precisaram de servidor nenhum. As plataformas entram quando é preciso compartilhar e revisar.

A diferença entre GitLab e GitHub no dia a dia é pequena: os dois hospedam repositórios Git e permitem revisão. Os nomes mudam um pouco (o GitHub chama o Merge Request de *Pull Request*). A diferença que mais importa para uma instituição é a próxima seção.

3.2 GITLAB SELF-MANAGED

O GitLab tem uma modalidade que a organização **instala e administra na própria infraestrutura**, chamada **GitLab Self-Managed**. A instituição mantém a sua instância nos seus servidores ou em infraestrutura autorizada, sem mandar o código para fora.



Isso dá à instituição mais controle sobre armazenamento dos dados, código-fonte, histórico, usuários, permissões, autenticação, auditoria, políticas de segurança e integração com outros sistemas.

⚠ Dentro de casa não é sinônimo de seguro

Não interprete Self-Managed como “se está dentro da instituição, está automaticamente seguro”. A segurança continua dependendo de configuração, controle de acesso, atualização do software, autenticação, política de senhas, backups, firewall, segmentação de rede, monitoramento, auditoria e gestão de vulnerabilidades. O ganho é outro: a instituição **mantém o ambiente sob a própria administração**.

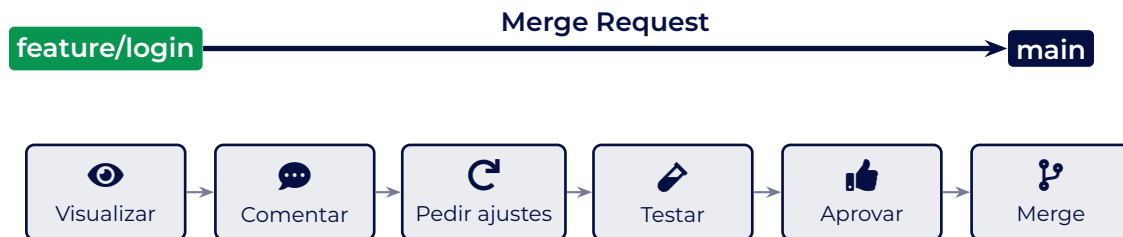
3.3 BRANCH PROTEGIDA

No GitLab, uma branch pode ser **protegida**. Numa branch protegida, ninguém faz `git push` direto (ou só quem tem permissão), e a única forma de mudar o conteúdo é por **Merge Request**. É comum a `main` ser protegida, para que nada chegue à versão principal sem revisão.

3.4 MERGE REQUEST

📖 Merge Request (MR)

É uma **solicitação para incorporar as alterações de uma branch em outra**, normalmente acompanhada de revisão. No GitHub, o mesmo recurso se chama *Pull Request*.



O fluxo é: a pessoa trabalha numa branch, envia a branch ao GitLab com `git push` e abre um Merge Request. A equipe então visualiza as alterações, revisa o código, comenta, pede ajustes, roda os testes automáticos (se houver pipeline), aprova e, por fim, faz o merge.

💬 Commit, branch e Merge Request

Commit registra uma alteração. **Branch** cria uma linha de desenvolvimento. **Merge Request** pede que essa linha seja integrada, como um processo que só segue depois do “de acordo” de quem tem a competência para aprovar.

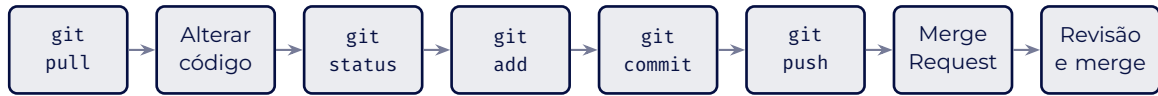
3.4.1 Abrindo um Merge Request pela interface web

Os nomes dos botões podem variar um pouco conforme a versão do GitLab instalada:

1. envie a sua branch: `git push origin minha-branch`;
2. abra o projeto no GitLab. Logo após o push, ele costuma mostrar um aviso com o botão **Create merge request**;
3. se o aviso não aparecer, vá em **Code > Merge requests > New merge request** e escolha a branch de origem (a sua) e a de destino (em geral, a main);
4. preencha um título claro e uma descrição do que mudou e por quê;
5. indique revisores, se a equipe usar;
6. clique em **Create merge request**.

Depois disso, a aba **Changes** mostra linha a linha o que mudou, e a revisão acontece em comentários ali mesmo.

3.5 O FLUXO BÁSICO DE TRABALHO EM EQUIPE



>_ TERMINAL

```
git switch main
git pull                    # começa sempre atualizado
git switch -c minha-branch  # trabalha numa branch própria
# ... realizar as alterações ...
git status
git add arquivo-alterado.py
git commit -m "Adiciona nova funcionalidade"
git push origin minha-branch
# depois: abrir o Merge Request no GitLab
```

💡 Regra de ouro

Comece sempre com `git pull` e mantenha a sua branch curta. Quanto mais tempo ela fica longe da main, maior a chance de conflito.

⚠️ `git add .`

`git add .` prepara **tudo** o que mudou na pasta, inclusive arquivos que você não queria enviar. Confira com `git status` antes, ou adicione os arquivos pelo nome.

4

GITLAB CLI: O GITLAB PELO TERMINAL

📖 GitLab CLI

O GitLab CLI, usado pelo comando `glab`, é a ferramenta oficial para **interagir com o GitLab diretamente pelo terminal**: Merge Requests, Issues, projetos, pipelines, branches, releases e autenticação.

Git

controle de versão

GitLab

plataforma de colaboração

glab

o GitLab no terminal

4.1 INSTALAÇÃO

As formas de instalar mudam conforme o sistema e a versão. Siga a página oficial: <https://gitlab.com/gitlab-org/cli>, seção *Installation*. Para conferir:

>_ TERMINAL

```
glab --version
```

4.2 AUTENTICAÇÃO

O `glab` precisa saber em qual GitLab e com qual usuário trabalhar. Num GitLab instalado pela instituição, informe o endereço dele:

>_ TERMINAL

```
glab auth login --hostname <endereço-do-gitlab>
```

O comando pergunta como você quer se autenticar. O caminho mais comum é um **token de acesso pessoal**, gerado no GitLab em **Preferências do usuário > Access tokens**.

⚠ Token é senha

Um token de acesso dá acesso à sua conta. Não compartilhe, não envie por e-mail

ou chat e não grave em arquivo de projeto. Dê a ele só as permissões necessárias e uma data de validade.

4.3 OS COMANDOS MAIS ÚTEIS

<code>glab mr create</code>	cria um Merge Request a partir da branch atual
<code>glab mr create -s origem -b destino</code>	escolhe as branches de origem e de destino
<code>glab mr list</code>	lista os Merge Requests abertos do projeto
<code>glab mr view</code>	mostra o Merge Request da branch atual
<code>glab mr diff</code>	mostra as alterações do Merge Request
<code>glab mr approve</code>	aprova um Merge Request (se você tiver permissão)
<code>glab mr merge</code>	faz o merge (se você tiver permissão)
<code>glab issue list</code>	lista as Issues
<code>glab ci status</code>	mostra a situação do pipeline da branch atual
<code>glab repo clone grupo/projeto</code>	clona um projeto pelo nome

Exemplo completo de criação de Merge Request:

>_ TERMINAL

```
glab mr create --source-branch desenvolvimento --target-branch main \
  --title "Integra o sistema ao repositório institucional" \
  --description "Une o histórico local ao histórico do GitLab."
```

Todo comando tem ajuda embutida: `glab mr create --help`.

4.4 TERMINAL OU INTERFACE WEB?

Tudo o que é versionamento e merge dá para fazer no Git, localmente. A parte de colaboração no GitLab dá para fazer com o **glab**. Mesmo assim, em ambiente institucional a **interface web continua importante** para revisão visual, aprovação, auditoria, acompanhamento dos pipelines e conversa da equipe.

⚠ As regras valem no terminal também

O **glab** não pula regras. Se a `main` exige aprovação de um responsável, `glab mr merge` vai esperar essa aprovação como a interface web esperaria.

5

PADRÕES DE COMMIT

No capítulo 2 vimos que um commit leva uma mensagem. Este capítulo trata de **como escrever essa mensagem** de um jeito que a equipe inteira (e as ferramentas) consigam ler.

5.1 POR QUE PADRONIZAR

Compare dois históricos do mesmo projeto:

SAÍDA · SEM PADRÃO

```
a41aded ajustes  
c2524a8 arrumei  
afe16bb agora vai  
4203776 mudanças do joão
```

SAÍDA · COM PADRÃO

```
a41aded fix(login): corrige senha vazia  
c2524a8 feat(relatorio): exporta em PDF  
afe16bb docs: explica a instalação  
4203776 test(login): cobre senha vazia
```

No segundo, dá para saber **o tipo** da mudança, **onde** ela aconteceu e **o que** foi feito, sem abrir um arquivo sequer. Um padrão de commit também permite automatizar tarefas: gerar notas de versão, decidir o próximo número de versão e barrar mensagens fora do combinado.

📖 A regra mais importante

O melhor padrão é **o que a equipe combinou e segue**. Os padrões deste capítulo são os mais usados, mas consistência vale mais que qualquer escolha isolada.

5.2 CONVENTIONAL COMMITS

O padrão mais difundido é o **Conventional Commits** 1.0.0 (<https://www.conventionalcommits.org/pt-br/v1.0.0/>). A estrutura é:

>_ TERMINAL · ESTRUTURA

```
<tipo>[escopo opcional]: <descrição>

[corpo opcional]

[rodapé(s) opcional(is)]
```

As regras principais da especificação:

- o **tipo** é obrigatório e vem primeiro, seguido de **dois-pontos e espaço**;
- o **escopo** é opcional: um substantivo entre parênteses que diz qual parte do sistema mudou, como `fix(parser)`;;
- a **descrição** vem logo depois de `:` e resume a mudança;
- o **corpo** é opcional e começa **uma linha em branco** depois da descrição, com quantos parágrafos forem necessários;
- os **rodapés** são opcionais e vêm uma linha em branco depois do corpo, no formato `Token: valor` (por exemplo, `Refs: #123` ou `Reviewed-by: Maria`);
- uma **quebra de compatibilidade** (*breaking change*) é indicada com `!` antes dos dois-pontos, com um rodapé `BREAKING CHANGE: <explicação>`, ou com os dois.

>_ TERMINAL · EXEMPLO COMPLETO

```
feat(api)!: exige autenticação em todos os endpoints

Antes, os endpoints de consulta eram públicos. Agora todo pedido
precisa do cabeçalho Authorization com um token válido.

BREAKING CHANGE: clientes sem token passam a receber 401.
Refs: #42
```

5.2.1 Os tipos

A especificação só define dois tipos: **feat** (novo recurso) e **fix** (correção de bug). Os outros são convenção, herdada das regras do projeto Angular. A lista abaixo é a aceita pela configuração padrão do *commitlint* (seção 5.6):

Tipo	Quando usar
feat	um recurso novo para quem usa o sistema
fix	a correção de um bug
docs	só documentação (README, comentários, manuais)
style	formatação que não muda o comportamento (espaços, ponto e vírgula, indentação)
refactor	reorganização do código sem mudar o que ele faz nem corrigir bug
perf	melhoria de desempenho
test	criação ou ajuste de testes
build	sistema de build ou dependências (npm, pip, Maven...)
ci	configuração de integração contínua (pipelines do GitLab, GitHub Actions)
chore	tarefas de manutenção que não se encaixam nos outros tipos
revert	desfaz um commit anterior

💡 Tipos e versões

Com Conventional Commits, ferramentas conseguem calcular a próxima versão pelo *Versionamento Semântico*: fix sobe o número de correção (1.2.**3**), feat sobe o número de recurso (1.**3**.0) e um BREAKING CHANGE sobe o número principal (**2**.0.0).

⚠️ Tipos fora da lista

Algumas referências populares em português acrescentam tipos como raw, cleanup, remove e init. Eles não fazem parte da especificação nem da configuração padrão do commitlint, que os rejeita. Só use se a equipe combinar e configurar isso.

5.3 BOAS PRÁTICAS PARA A MENSAGEM

- **Uma mudança por commit.** Se a descrição precisa de um “e”, provavelmente são dois commits. Commits pequenos são mais fáceis de revisar e de desfazer.
- **Descrição curta.** O livro *Pro Git* recomenda uma primeira linha de até uns 50 caracteres e o corpo quebrado em 72 colunas. A configuração padrão do commitlint aceita até 100 caracteres no cabeçalho.
- **O corpo explica o porquê.** O “o quê” está no `git diff`. O motivo da mudança só existe na mensagem.
- **Um único modo verbal.** As convenções em inglês usam o imperativo (“add”, “fix”). Em português, equipes usam o imperativo (“corrigir”), o presente (“corrige”) ou o gerúndio (“corrigindo”). A especificação não exige nenhum. Escolha um e man-

tenha.

- **Idioma combinado.** A especificação também não fixa idioma. Misturar português e inglês no mesmo histórico atrapalha a busca.

5.4 GITMOJI: EMOJIS NAS MENSAGENS

O **gitmoji** (<https://gitmoji.dev>) associa um emoji a cada tipo de mudança. No texto do commit, o emoji aparece como um código entre dois-pontos, o *shortcode*. Alguns dos mais usados, com a descrição oficial traduzida:

Código	Significado	Código	Significado
:sparkles:	novo recurso	:bug:	corrige um bug
:memo:	documentação	:art:	estrutura ou formato do código
:zap:	desempenho	:recycle:	refatoração
:fire:	remove código ou arquivos	:truck:	move ou renomeia arquivos
:white_check_mark:	testes	:construction_worker:	CI
:wrench:	arquivos de configuração	:package:	arquivos compilados ou pacotes
:lock:	segurança ou privacidade	:ambulance:	correção crítica urgente
:pencil2:	corrige erro de digitação	:rewind:	reverte mudanças
:tada:	começa um projeto	:bookmark:	versão ou tag
:page_facing_up:	licença	:card_file_box:	mudanças de banco de dados

A lista completa tem 75 emojis e está em <https://gitmoji.dev>.

⚠ Três cuidados com emojis

1. **O terminal não converte.** O GitHub mostra o emoji na página do commit, mas o `git log` exibe o texto `:sparkles:` como foi escrito.
2. **O mesmo emoji muda de sentido entre fontes.** Na lista oficial, `:boom:` é quebra de compatibilidade, mas há tutoriais que o usam para “reverter”. Siga a lista oficial.
3. **Emoji antes do tipo quebra o padrão.** A especificação exige que a mensagem comece pelo tipo. Uma mensagem como `:bug: fix: corrige o login` é rejeitada pelo `commitlint` na configuração padrão.

5.5 UM PADRÃO COMBINANDO OS DOIS

Muitas equipes juntam gitmoji e Conventional Commits num padrão próprio. É o caso do repositório pessoal do autor desta apostila, que usa:

>_ TERMINAL · PADRÃO DE UMA EQUIPE

```
<:emoji:> <tipo>(<escopo>): <Descrição no gerúndio>

:memo: docs(capacitacao): Adicionando os exercícios de assinatura
:bug: fix(login): Corrigindo o acesso com senha vazia
:truck: chore(docs): Movendo o manual para a pasta materials
```

Não há nada de errado nisso, desde que **seja uma decisão da equipe, esteja escrita** (no README ou num guia de contribuição) e **as ferramentas estejam configuradas para aceitá-la**. O ponto é a consistência.

5.6 FERRAMENTAS

commitlint: barrar mensagens fora do padrão

O **commitlint** (<https://commitlint.js.org>) confere cada mensagem antes de o commit ser gravado. Num projeto com Node.js:

>_ TERMINAL

```
npm install -D @commitlint/cli @commitlint/config-conventional
echo "export default { extends: ['@commitlint/config-conventional'] };" >
  commitlint.config.js

# testar uma mensagem sem precisar fazer commit
echo "feat: adiciona exportação em PDF" | npx commitlint
```

Para rodar automaticamente em todo `git commit`, o guia oficial usa o *husky*:

>_ TERMINAL

```
npm install --save-dev husky
npx husky init
echo "npx --no -- commitlint --edit \"$1" > .husky/commit-msg
```

Depois disso, `git commit -m "foo: teste"` é recusado, porque `foo` não é um tipo válido.

⚠ A configuração padrão é rígida

Além de exigir o tipo no início, a configuração padrão rejeita descrições que começam com letra maiúscula. Um padrão como o da seção anterior precisa de regras

próprias no `commitlint.config.js`.

gitmoji-cli: escolher o emoji num menu

>_ TERMINAL

```
npm i -g gitmoji-cli
gitmoji -c          # monta o commit de forma interativa
gitmoji -l          # lista os emojis
```

Sem ferramenta nenhuma

Nada disso é obrigatório. Um arquivo `CONTRIBUTING.md` com o padrão e três exemplos já resolve boa parte do problema, e a revisão do Merge Request (capítulo 3) é um bom momento para lembrar o combinado.

6

ASSINATURA DE COMMITS COM GPG E SSH

6.1 O PROBLEMA: O AUTOR DE UM COMMIT É SÓ UM TEXTO

No capítulo 1 configuramos o nome e o e-mail com `git config`. O Git aceita **qualquer** valor ali, sem conferir nada. Veja:

>_ TERMINAL

```
git -c user.name="Linus Torvalds" -c user.email="torvalds@linux-foundation.org" \  
  commit --allow-empty -m "Commit que parece do Linus"  
git log --format='%an <%ae>' -1
```

SAÍDA

```
Linus Torvalds <torvalds@linux-foundation.org>
```

O commit foi feito por nós, mas o histórico diz que foi o criador do Git. Se esse commit for enviado ao GitHub, ele aparece com o nome e a foto de quem tem aquele e-mail cadastrado.

Assinatura de commit

Assinar um commit é anexar a ele uma **assinatura criptográfica** feita com uma chave privada que só você tem. Quem tem a sua chave **pública** consegue conferir que o commit veio de você e que não foi alterado depois. O GitHub e o GitLab fazem essa conferência e mostram um selo ao lado do commit.

Os selos

Selo	Significado
Verified	o commit está assinado e a assinatura foi conferida com uma chave cadastrada na conta
Unverified	o commit está assinado, mas a assinatura não pôde ser conferida (chave não cadastrada, e-mail diferente, chave de outra pessoa)
(sem selo)	o commit não foi assinado

⚠ O e-mail precisa bater

Para o selo **Verified**, o e-mail do commit (`git config user.email`) precisa ser um e-mail **verificado** na sua conta do GitHub ou do GitLab. No caso do GPG, ele também precisa estar dentro da chave. Essa é a causa mais comum de “Unverified”.

6.2 GPG OU SSH?

Há dois jeitos de assinar, e os dois são aceitos pelo GitHub e pelo GitLab:

	GPG	SSH
Ferramenta	GnuPG (gpg)	OpenSSH (ssh-keygen), que já vem com o Git
Requisito	GnuPG instalado	Git 2.34 ou mais novo
Chave	uma chave própria, que carrega nome e e-mail	a mesma chave SSH que você usa para acessar o repositório pode servir
Validade	pode ter data de expiração embutida	a validade é definida no cadastro do GitLab
Conferir no terminal	direto, com o chaveiro do GPG	precisa de um arquivo <code>allowed_signers</code>
Quando escolher	padrão consolidado, usado há muito tempo	mais simples para quem já usa SSH

💡 Dica

Escolha **um** dos dois e siga só a seção correspondente. Se não souber qual, comece pelo SSH: são menos passos.

6.3 ASSINANDO COM GPG

6.3.1 Instalar o GnuPG

- **Linux:** costuma vir instalado. Confira com `gpg --version`. No Debian e no Ubuntu, se faltar: `sudo apt install gnupg`.
- **Windows:** o Git for Windows já traz um `gpg` dentro do **Git Bash**. Use o Git Bash para todos os comandos desta seção.

⚠ Windows: um GPG só

Se você também instalar o Gpg4win, passa a ter **dois** GPGs no computador, cada um com o próprio chaveiro. Uma chave criada num não aparece no outro, e o Git

reclama de “No secret key”. Crie a chave e assine sempre pelo mesmo. Para saber qual o Git está usando, rode `which gpg` no Git Bash.

6.3.2 Passo 1 — Ver se você já tem uma chave

>_ TERMINAL

```
gpg --list-secret-keys --keyid-format=long
```

Se aparecer uma chave com o seu e-mail, pule para o passo 3.

6.3.3 Passo 2 — Gerar a chave

>_ TERMINAL

```
gpg --full-generate-key
```

O comando faz algumas perguntas:

Pergunta	O que responder
Tipo de chave	aperte <code>[Enter]</code> para aceitar o padrão. Nas versões novas é ECC (ed25519); nas antigas, “RSA e RSA”. O GitHub e o GitLab aceitam os dois
Tamanho	só aparece para RSA: digite 4096, como recomenda o GitLab
Validade	0 = não expira, que é o padrão recomendado pelo GitHub. Uma data (como 1y, um ano) é mais segura, mas exige renovar a chave
Nome e e-mail	o seu nome e o mesmo e-mail verificado no GitHub ou no GitLab
Senha	uma senha forte. Ela protege a chave se alguém copiar o seu computador

6.3.4 Passo 3 — Descobrir o ID da chave

>_ TERMINAL

```
gpg --list-secret-keys --keyid-format=long
```

SAÍDA

```
sec  rsa4096/30F2B65B9246B6CA 2017-08-18 [SC]
    D5E4F29F3275DC0CDA8FFC8730F2B65B9246B6CA
uid                          [ultimate] Maria Exemplo <maria@exemplo.com>
ssb  rsa4096/B7ABC0813E4028C0 2017-08-18 [E]
```

O ID é o que vem **depois da barra** na linha sec: aqui, 30F2B65B9246B6CA. Numa chave ECC, a linha começa com ed25519/. O seu ID será outro.

6.3.5 Passo 4 — Exportar a chave pública

>_ TERMINAL

```
gpg --armor --export 30F2B65B9246B6CA
```

Copie **tudo**, da linha -----BEGIN PGP PUBLIC KEY BLOCK----- até a linha -----END PGP PUBLIC KEY BLOCK-----, incluindo as duas.

⚠ Pública, nunca privada

O que vai para o site é a chave **pública** (--export). Nunca use --export-secret-keys para isso. A chave privada não sai do seu computador.

6.3.6 Passo 5 — Cadastrar no GitHub

1. Clique na sua foto (canto superior direito) > **Settings**.
2. Na seção **Access**, clique em **SSH and GPG keys**.
3. Clique em **New GPG key**.
4. Em **Title**, dê um nome (por exemplo, “Notebook pessoal”). Em **Key**, cole o bloco copiado.
5. Clique em **Add GPG key**. O GitHub pode pedir a sua senha de novo.

6.3.7 Passo 5 (alternativo) — Cadastrar no GitLab

1. Clique no seu avatar (canto superior direito) > **Edit profile**.
2. Na barra lateral, em **Access**, clique em **GPG keys**.
3. Clique em **Add new key**, cole o bloco em **Key** e clique em **Add key**.

Depois de cadastrada, a chave não pode ser editada: para mudar, remova e cadastre de novo. Num GitLab instalado pela instituição, o caminho é o mesmo.

6.3.8 Passo 6 — Dizer ao Git qual chave usar

>_ TERMINAL

```
git config --global user.signingkey 30F2B65B9246B6CA
git config --global commit.gpgsign true           # assina todo commit
git config --global tag.gpgsign true               # assina toda tag
```

💡 Já usou assinatura SSH antes?

Se algum dia configurou `gpg.format ssh`, desfaça com `git config --global --unset gpg.format`. Sem isso, o Git continua tentando assinar com SSH.

6.3.9 Passo 7 — Ensinar o GPG a pedir a senha no terminal

No Linux e no Git Bash, o GPG precisa saber em qual terminal pedir a senha da chave. Rode uma vez:

>_ TERMINAL

```
[ -f ~/.bashrc ] && echo -e '\nexport GPG_TTY=$(tty)' >> ~/.bashrc
source ~/.bashrc
```

Quem usa **zsh** coloca a linha `export GPG_TTY=$(tty)` no `~/.zshrc`.

6.3.10 Passo 8 — Testar

>_ TERMINAL

```
git commit --allow-empty -m "Testando a assinatura"
git log --show-signature -1
```

SAÍDA

```
commit e55fbf5be5b86eb89737adea712d8c605b5f9529
gpg: Assinatura feita seg 28 set 2026 14:51:16 -03
gpg:          usando RSA chave 0D9FFCDB3C4EBDC3E7225C8CB4BEA86F1F6C7956
gpg: Assinatura correta de "Maria Exemplo <maria@exemplo.com>" [final]
Author: Maria Exemplo <maria@exemplo.com>
```

A linha “Assinatura correta” (em inglês, “Good signature”) confirma que funcionou. Se você não ligou o `commit.gpgsign`, assine um commit específico com `git commit -S -m "..."`.

6.4 ASSINANDO COM SSH

6.4.1 Requisitos

- **Git 2.34 ou mais novo** (`git --version`);
- **OpenSSH 8.1 ou mais novo** (`ssh -V`). A versão 8.7 tem a assinatura quebrada: nela, atualize para a 8.8 ou mais nova;
- no GitLab, versão 15.7 ou mais nova. Num GitLab da instituição, confirme a versão com o administrador.

6.4.2 Passo 1 — Ter uma chave SSH

Se você já acessa o GitHub ou o GitLab por SSH, já tem uma: normalmente `~/.ssh/id_ed25519.pub`. Se não tiver, crie:

>_ TERMINAL

```
ssh-keygen -t ed25519 -C "maria@exemplo.com"
```

Aperte `[Enter]` para aceitar o local padrão e escolha uma senha. O arquivo terminado em `.pub` é a chave **pública**. O outro, sem extensão, é a **privada**, e nunca sai do seu computador.

6.4.3 Passo 2 — Dizer ao Git para assinar com SSH

>_ TERMINAL

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgSign true
```

6.4.4 Passo 3 — Cadastrar como chave de assinatura no GitHub

1. Foto > **Settings** > **SSH and GPG keys** > **New SSH key**.
2. Em **Title**, dê um nome. Em **Key type**, escolha **Signing Key**.
3. Em **Key**, cole o conteúdo do arquivo `.pub` (`cat ~/.ssh/id_ed25519.pub`) e clique em **Add SSH key**.

⚠ No GitHub, a mesma chave entra duas vezes

Uma chave cadastrada como *Authentication Key* serve para `git push`, mas **não** para verificar assinaturas. Se quiser usar a mesma chave para as duas coisas, cadastre-a duas vezes: uma com cada tipo.

Pelo terminal, com o GitHub CLI, dá no mesmo. A opção `--type` existe nas versões do `gh` lançadas a partir de 2023. Se a sua recusar a opção, use a página web.

>_ TERMINAL

```
gh ssh-key add ~/.ssh/id_ed25519.pub --type signing --title "Notebook pessoal"
```

6.4.5 Passo 3 (alternativo) — Cadastrar no GitLab

1. Avatar > **Edit profile** > **Access** > **SSH keys** > **Add new key**.
2. Cole a chave pública em **Key** e dê um **Title**.
3. Em **Usage type**, deixe **Authentication & Signing** (o padrão) ou escolha **Signing**. Com **Authentication** sozinho, os commits ficam “Unverified”.
4. Opcionalmente, defina uma **Expiration date** e clique em **Add key**.

6.4.6 Passo 4 — Permitir a conferência no seu computador

O GitHub e o GitLab conferem a assinatura sozinhos. Mas, para o *seu* Git conferir, ele precisa de uma lista de chaves confiáveis. Sem ela, aparece:

```
SAÍDA

error: gpg.ssh.allowedSignersFile needs to be configured and exist for ssh
signature verification
```

Crie a lista uma vez:

```
TERMINAL

touch ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
echo "$(git config --get user.email) namespaces=\"git\" $(cat
~/.ssh/id_ed25519.pub)" >> ~/.ssh/allowed_signers
```

Cada linha do arquivo associa um e-mail a uma chave. O `namespaces="git"` limita a chave a assinaturas do Git.

6.4.7 Passo 5 — Testar

```
TERMINAL

git commit --allow-empty -m "Testando a assinatura SSH"
git log --show-signature -1
```

```
SAÍDA

commit 86613eb51dee51c80a61f2cf8f1532df427e60c5
Good "git" signature for maria@exemplo.com with ED25519 key
SHA256:Z/dW7mOFmUlo/Po1CSObYXo03bAPY2bTohQtnJY0PGc
Author: Maria Exemplo <maria@exemplo.com>
```

6.5 CONFERINDO NO GITHUB E NO GITLAB

Envie um commit assinado com `git push` e abra a lista de commits:

- **GitHub:** na aba **Commits** do repositório, o selo **Verified** aparece ao lado do commit. Clique nele para ver qual chave assinou.
- **GitLab:** em **Code > Commits**, o selo **Verified** aparece da mesma forma.

No terminal, uma letra resume o estado de cada commit:

>_ TERMINAL

```
git log --format='%h %G? %s'
```

-
- G** assinatura válida
 - N** sem assinatura
 - B** assinatura inválida
 - E** não foi possível conferir (por exemplo, falta a chave pública ou o `allowed_signers`)
-

💡 Vigilant mode no GitHub

Em **Settings > SSH and GPG keys**, a opção **Vigilant mode** (“Flag unsigned commits as unverified”) faz o GitHub marcar como **Unverified** todo commit seu que não esteja assinado, inclusive os que alguém fizer se passando por você. Ligue só depois que todos os seus computadores estiverem assinando.

6.6 CUIDADOS COM A CHAVE

- **Guarde a chave privada e a senha.** Se perder a chave, é só criar outra e cadastrar, mas perde-se a continuidade.
- **Chave vazou? Revoke e cadastre outra.** Aqui as plataformas diferem:
 - ▶ no **GitHub**, a verificação é registrada no momento do push. Os commits antigos **continuam Verified** mesmo que a chave seja removida, expire ou seja revogada;
 - ▶ no **GitLab**, **Revoke** torna **Unverified** os commits antigos assinados com a chave, e **Remove** os mantém verificados. Use *Revoke* quando a chave vazou.
- **Commits feitos pela interface web** já saem assinados pela própria plataforma.
- No GitHub, um Pull Request integrado com **Rebase and merge** reescreve os commits, e eles perdem a sua assinatura.
- **Não é preciso publicar a chave em servidor de chaves** (*keyserver*). Alguns tutoriais pedem isso, mas o GitHub e o GitLab só usam a chave cadastrada na conta.

6.7 OUTRAS PLATAFORMAS E FERRAMENTAS

- **Bitbucket Cloud:** aceita GPG e SSH. A chave GPG entra em **Settings > Personal Bitbucket settings > GPG keys**. Para SSH, serve a mesma chave pessoal de acesso. Só commits enviados pela linha de comando são verificados.

- **1Password:** guarda a chave SSH no cofre e configura o Git sozinho (`gpg.format`, `user.signingkey`, `commit.gpgsign` e o programa de assinatura dele). A chave continua precisando ser cadastrada como *Signing Key* no GitHub ou com *Usage type* de assinatura no GitLab.

6.8 PROBLEMAS COMUNS

Mensagem ou sintoma	O que fazer
<code>error: gpg failed to sign the data</code>	quase sempre é o <code>GPG_TTY</code> (passo 7 do GPG). Confira também se o <code>user.signingkey</code> bate com <code>gpg --list-secret-keys</code>
<code>gpg: signing failed: Inappropriate ioctl for device</code>	falta o <code>export GPG_TTY=\$(tty)</code>
<code>No secret key / secret key not available</code>	o ID configurado não existe neste chaveiro. No Windows, confira se o Git usa o mesmo GPG em que a chave foi criada
<code>gpg.ssh.allowedSignersFile</code> needs to be configured and exist for ssh signature verification	crie o <code>allowed_signers</code> (passo 4 do SSH). A assinatura em si está certa
either <code>user.signingkey</code> or <code>gpg.ssh.defaultKeyCommand</code> needs to be configured	falta o <code>git config --global user.signingkey</code>
Selo Unverified no site	o e-mail do commit não está verificado na conta, a chave não foi cadastrada, ou a chave SSH foi cadastrada só para autenticação

💡 Fontes oficiais

Os passos deste capítulo seguem as documentações do GitHub (<https://docs.github.com/pt/authentication/managing-commit-signature-verification>) e do GitLab (https://docs.gitlab.com/user/project/repository/signed_commits/). Os nomes dos botões podem mudar com novas versões das plataformas. Se algo estiver diferente na sua tela, confira lá.

7

PROBLEMA 1: INTEGRAR UM SISTEMA JÁ VERSIONADO AO GITLAB INSTITUCIONAL

Agora saímos da teoria. Este é um problema real, enfrentado no estágio.

7.1 O CENÁRIO

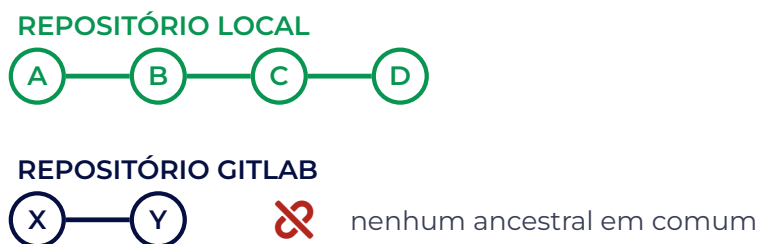
A instituição tem um GitLab empresarial restrito. Ao criar um repositório novo, ele já vem com:

- uma branch `main` criada;
- um `README.md` padrão;
- regras institucionais;
- a `main` **protegida**: não aceita `git push` direto, só Merge Request.

Só que vários sistemas **já existem** nos computadores, e alguns já tinham anos de histórico Git próprio. Era preciso levar esses sistemas para o GitLab **sem perder o histórico**.

7.2 POR QUE NÃO FUNCIONA DO JEITO ÓBVIO

São dois históricos que nasceram separados:



O Git sabe juntar linhas que **nasceram de um mesmo ponto**. Entre A--B--C--D e X--Y não existe esse ponto. Por isso, um merge comum é recusado:

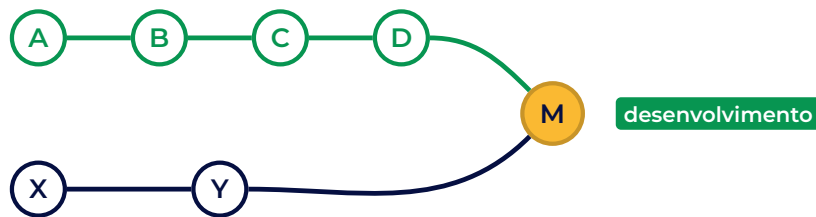
SAÍDA

```
fatal: refusing to merge unrelated histories
```

E um `git push` direto para a `main` também é recusado, porque ela é protegida.

7.3 O OBJETIVO

Transformar os dois históricos num só, sem perder nenhum commit:



O commit **M** é um commit de merge que une os dois históricos. Depois dele, a branch `desenvolvimento` contém a `main` institucional, então um Merge Request para a `main` passa a funcionar normalmente.

7.4 A SOLUÇÃO, PASSO A PASSO

Passo 1 — Adicionar o repositório remoto

>_ TERMINAL

```
git remote add origin <URL_DO_REPOSITORIO>
git remote -v
```

SAÍDA

```
origin    <URL_DO_REPOSITORIO> (fetch)
origin    <URL_DO_REPOSITORIO> (push)
```

Isso dá ao Git local um endereço remoto, chamado `origin`. A URL aparece no botão **Code** (ou **Clone**) da página do projeto no GitLab.

Passo 2 — Buscar as informações do remoto

>_ TERMINAL

```
git fetch origin
```

SAÍDA

```
From <URL_DO_REPOSITORIO>  
* [new branch]      main      -> origin/main
```

O fetch baixa o histórico do GitLab para uma referência de leitura chamada origin/main, **sem mexer** na sua branch nem nos seus arquivos.

Passo 3 — Renomear a branch local

>_ TERMINAL

```
git branch -m main desenvolvimento
```

A sua main local vira desenvolvimento. Isso evita confundi-la com a main institucional, que é protegida e não pode receber push. Agora existem:

- | | |
|------------------------|---------------------------------------|
| desenvolvimento | o histórico original do sistema (A–D) |
| origin/main | o histórico institucional (X–Y) |

Dica

Se a sua branch atual não se chama main, use só `git branch -m desenvolvimento`, que renomeia a branch em que você está.

Passo 4 — Unir os históricos

>_ TERMINAL

```
git merge --allow-unrelated-histories origin/main
```

O parâmetro `--allow-unrelated-histories` deve ser usado com consciência.

✗ Ele **NÃO** significa
“ignore todos os conflitos”.

✓ Ele **significa**
“eu sei que esses históricos não têm
ancestral comum e autorizo o Git a
tentar integrá-los”.

Como os dois lados têm um README.md diferente, surge um conflito:

SAÍDA

```
Mesclagem automática de README.md
CONFLITO (adicionar/adicionar): conflito de mesclagem em README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Passo 5 — Resolver o conflito

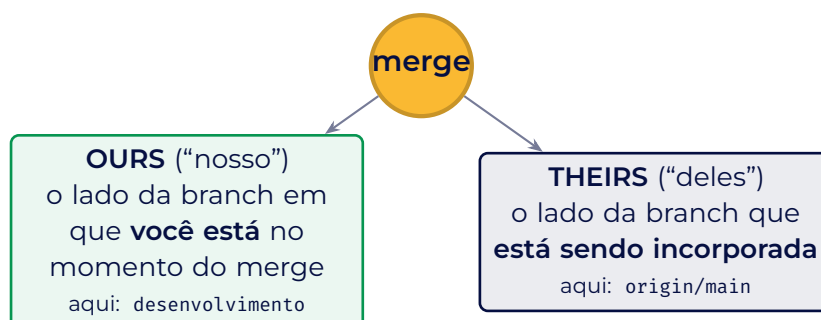
TERMINAL

```
git status
```

SAÍDA

```
Caminhos não mesclados:
(usar "git add <arquivo>..." para marcar resolução)
ambos adicionaram:  README.md
```

Aqui entra a parte que mais causa erro: **ours** e **theirs**.



No nosso caso, queríamos manter o README do sistema, que está na branch em que estamos. Então:

TERMINAL

```
git restore --ours README.md
# forma tradicional, com o mesmo efeito:
```

>_ TERMINAL

```
git checkout --ours README.md
```

Se quiséssemos manter o README institucional:

>_ TERMINAL

```
git restore --theirs README.md  
# ou: git checkout --theirs README.md
```

⚠️ **ours não é “o do meu computador”**

ours significa **o lado correspondente à branch atual durante aquele merge**. theirs é o outro lado. Os dois arquivos estão no seu computador. Interpretar errado faz você preservar a versão errada sem perceber.

⚠️ **No rebase, os papéis se invertem**

Durante um `git rebase`, ours passa a ser a branch *sobre a qual* você está reaplicando os commits, e theirs são os seus próprios commits. Nesta solução usamos merge, então vale a regra de cima. Se um dia usar rebase, confira antes.

💡 **E se eu quiser os dois READMEs?**

Você não precisa escolher um lado. Abra o `README.md`, junte à mão as partes que interessam dos dois, apague os marcadores <<<<<<, ===== e >>>>>>, e siga para o passo 6.

Passo 6 — Registrar a resolução

>_ TERMINAL

```
git add README.md  
git commit
```

Sem `-m`, o Git abre um editor com a mensagem “Merge remote-tracking branch 'origin/main' into desenvolvimento” já preenchida. Salve e feche. Este é o commit **M**: o histórico agora está integrado.

>_ TERMINAL

```
git log --oneline --graph --all
```

SAÍDA

```
* 89647ff Merge remote-tracking branch 'origin/main' into desenvolvimento
|\
| * 4738004 Initial commit
* 3ee51ff Ajuste
* 40d59e6 Primeira versão
```

Passo 7 — Publicar a nova branch

>_ TERMINAL

```
git push origin desenvolvimento
```

SAÍDA

```
* [new branch]      desenvolvimento -> desenvolvimento
```

No GitLab passam a existir, lado a lado, a main (intocada e protegida) e a desenvolvimento (o sistema com o histórico integrado).

Passo 8 — Abrir o Merge Request

Este é o ponto final de todo o processo. Abra um Merge Request de desenvolvimento para main, pela interface web (capítulo 3) ou pelo terminal:

>_ TERMINAL

```
glab mr create -s desenvolvimento -b main --title "Integra o sistema ao
repositório institucional"
```

Como os dois históricos agora são relacionados e o conflito já foi tratado localmente, o merge **não é mais recusado**. A aprovação fica com quem tem permissão no repositório, conforme as regras do projeto.

7.5 RESUMO EM OITO LINHAS

>_ TERMINAL

```
git remote add origin <URL_DO_REPOSITORIO>      # 1. conectar
git fetch origin                                  # 2. baixar
git branch -m main desenvolvimento                 # 3. renomear
git merge --allow-unrelated-histories origin/main # 4. unir
git restore --ours README.md                       # 5. decidir o conflito
```

>_ TERMINAL

```
git add README.md && git commit          # 6. registrar
git push origin desenvolvimento           # 7. publicar
glab mr create -s desenvolvimento -b main # 8. pedir a integração
```

7.6 ERROS QUE PODEM APARECER

Mensagem	O que fazer
fatal: refusing to merge unrelated histories	faltou o <code>--allow-unrelated-histories</code> no passo 4
error: remote origin already exists.	o projeto já tem um origin. Confira com <code>git remote -v</code> ; para trocar a URL, use <code>git remote set-url origin <URL></code>
remote: GitLab: You are not allowed to push code to protected branches on this project.	you tentou enviar para a main protegida. Envie a desenvolvimento e abra um Merge Request
error: Committing is not possible because you have unmerged files.	ainda há conflito sem resolver. Rode <code>git status</code> , resolva e faça <code>git add</code>

💡 Dica

Quer treinar isso sem servidor nenhum? O exercício 4.1 (o desafio) do capítulo 9 simula o GitLab institucional com uma pasta do seu computador e reproduz o caso inteiro.

8

PROBLEMA 2: DOCUMENTAR SISTEMAS LEGADOS COM UM AGENTE DE IA

8.1 VERSIONADO NÃO É DOCUMENTADO

Resolvido o Problema 1, os sistemas passaram a ter histórico no GitLab. Mas surgiu outro problema:

📖 A ideia central

Um sistema estar versionado não significa que ele esteja documentado ou compreendido.

Os sistemas existentes, alguns legados, em sua maioria não tinham documentação nenhuma. O quadro típico era este: documentação incompleta ou desatualizada, conhecimento concentrado em poucas pessoas, regras de negócio que só existiam no código, dependências desconhecidas, configurações pouco claras e manutenção difícil. Isso forma uma cadeia:

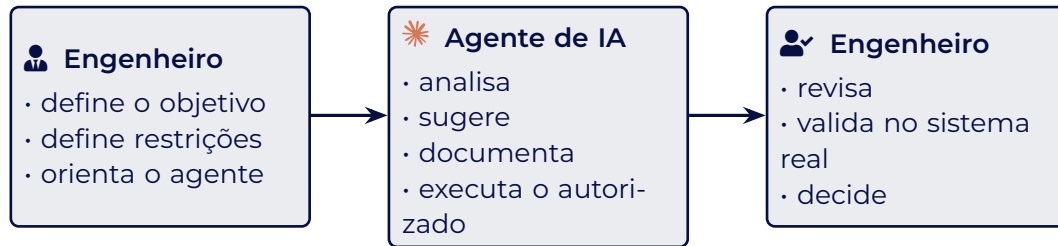


8.2 A ABORDAGEM: IA COMO FERRAMENTA DE ENGENHARIA

A ferramenta usada para acelerar o trabalho foi um **agente de IA para desenvolvimento**, o **Claude Code**, da Anthropic. Diferente de um chat comum, um agente deses trabalha dentro da pasta do projeto: lê os arquivos, roda comandos (com permissão) e escreve documentos.

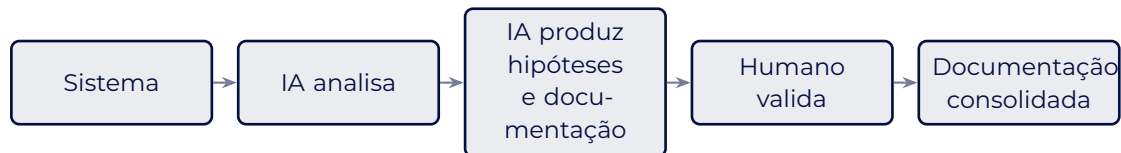
⚠️ A ideia não é “mandar a IA escrever a documentação”

A proposta é usar a IA como **ferramenta auxiliar de engenharia de software**: descobrir a arquitetura, analisar o código, identificar dependências, fluxos, regras de negócio, duplicações, problemas e riscos de segurança, sugerir refatorações e, por fim, produzir documentação.



8.3 A IA NÃO SUBSTITUI A VALIDAÇÃO HUMANA

O agente não é autoridade sobre o sistema. Ele pode interpretar errado regras de negócio, a intenção de quem escreveu o código, comportamentos implícitos, configurações, dependências e funcionalidades. Por isso o fluxo é sempre:



Regra

A documentação gerada por IA deve ser validada contra o sistema real. A responsabilidade final sobre as decisões técnicas continua sendo humana.

8.4 A EVOLUÇÃO REAL DOS PROMPTS

8.4.1 De onde vêm estes dados

Um **prompt** é a instrução que se dá ao agente. A qualidade do resultado depende muito dela. Para esta apostila, foi analisado o histórico de prompts do Claude Code do autor (o arquivo `~/claude/history.jsonl`), que registra cada pedido feito ao agente:

717 prompts registrados

123 sessões de trabalho

10/07 a 28/09/2026 período coberto

8 sistemas com trabalho de documentação completa (13/07 a 15/09)

Os nomes dos sistemas institucionais foram omitidos. Onde for preciso distingui-los, eles aparecem como Sistema A, B, C...

Limite dos dados

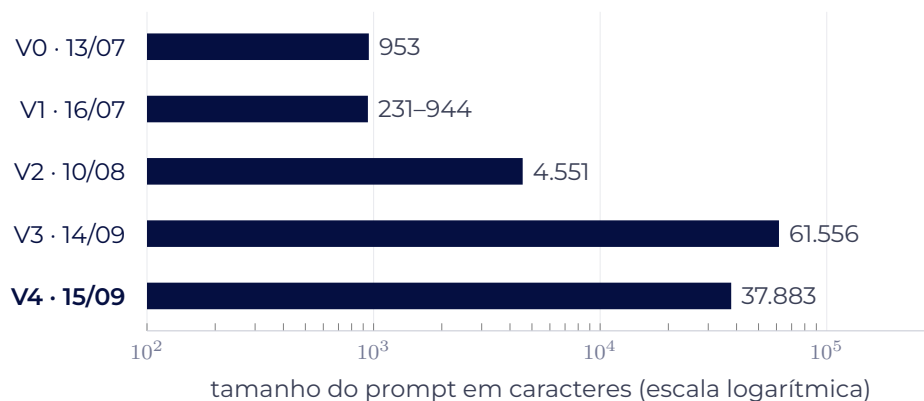
O histórico guarda sempre o texto digitado, mas o texto dos prompts *colados*

só ficou salvo a partir de 03/09. Dos prompts colados antes disso restam só o tamanho e a data. As respostas do agente não foram analisadas, só os pedidos.

8.4.2 Cinco versões

Versão	Início	Idioma	Tamanho	Características
V0	13/07	PT	953	Sequência de 9 prompts colados: engenharia reversa, arquitetura, especificações, tarefas. Já pedia para não modificar e descobrir antes de escrever
V1	16/07	PT	231–944	Pedidos curtos digitados, que cresciam a cada reuso. Em 24/07 surge o GAP em vez de inferir ; em 27/07, “ não altere nenhuma linha de código ”
V2	10/08	PT	4.551	Lista numerada de 47 itens: arquitetura, defeitos, LGPD (RIPD/DPIA), ADRs, UML, Docker, guia para leigos, projeto de evolução nos moldes do PMI
V3	14/09	EN	61.556	Protocolo de refatoração: fase 0 somente leitura , regra de não pressupor, política de commits. Sem exigência de idioma
V4	15/09	EN	37.883	Prompt final (model/SYSTEM_DOCUMENTATION.md): 42 seções, saída obrigatória em PT-BR , nunca inventar, AS-IS/TO-BE/GAP, <i>Safe Stop-and-Report</i> , fases com validação

Tamanho em caracteres. “Início” é a data do primeiro uso de cada versão.



Do primeiro pedido livre (231 caracteres) ao prompt final (37.883), o tamanho cresceu cerca de **164 vezes**. O V4 é **38 % menor** que o V3: ficou mais enxuto e, ao mesmo tempo, mais rigoroso.

8.4.3 Trechos que marcam a evolução

🗨️ V0 — 13/07, português

“Você é um Arquiteto de Software Sênior. Sua missão **NÃO** é modificar este projeto. [...] Faça uma engenharia reversa completa [...] Não escreva ainda a documentação final. Primeiro gere um relatório de descoberta [...] Se faltar contexto, continue investigando automaticamente.”

Já tinha a ideia de descobrir antes de escrever. Mas mandava o agente “continuar sozinho” quando faltasse contexto, e ainda não proibia inventar.

V1 — 24/07, português

“Se alguma informação estiver faltando, ela deve ser registrada como uma lacuna (GAP) em vez de ser inferida.”

Primeira vez que a falta de informação vira um registro explícito, e não um palpite.

V3 — 14/09, inglês

“You are NOT authorized to simply rewrite the entire system at once. [...] OBSERVE → INVENTORY → UNDERSTAND”

“Você NÃO está autorizado a simplesmente reescrever o sistema inteiro de uma vez. Observar → inventariar → entender.”

V4 — 15/09, inglês

“ALL CONTENT YOU PRODUCE MUST BE WRITTEN IN BRAZILIAN PORTUGUESE.”
“Never transform an assumption into a fact.”

“Todo o conteúdo que você produzir deve ser escrito em português brasileiro.” “Nunca transforme uma suposição em fato.”

8.5 O QUE DEU ERRADO E O QUE CADA FALHA ENSINOU

O histórico mostra os tropeços com clareza. Cada regra importante do prompt final apareceu **logo depois** de uma falha observada:

Falha observada	Ocorrências	Lição que virou regra
O agente parou no meio ou foi preciso pedir “continue de onde parou”	~27	Trabalho em fases menores, com artefato salvo ao fim de cada fase
Limite de uso ou de contexto atingido	14 + 10	Idem; prompt mais enxuto do V3 para o V4
Prompt reenviado ou reexecutado	4	Instruções mais precisas desde o início
Documentação superficial ou incompleta (“quero uma documentação mais profunda”)	4	Análise profunda em fases e auditoria de completude
Resposta em inglês depois do prompt em inglês	1	§1 do V4: saída obrigatória em PT-BR
Agente criou ou alterou algo que não devia	~5	Fase 0 somente leitura e <i>Safe Stop-and-Report</i>
Commits com coautoria de IA	4 lembretes	Cláusula explícita no protocolo de refatoração (V3)
Complexidade desnecessária (“preciso de coisas óbvias”)	2	Pedir clareza e público leigo

“14 + 10” = 14 avisos de limite de uso e 10 compactações de contexto. Contagens aproximadas, feitas sobre o texto dos prompts.

💡 Dica

Não houve nenhuma queixa explícita de informação inventada. A regra “nunca inventar” entrou de forma **preventiva**: primeiro como “GAP em vez de inferir” (24/07), depois como princípio absoluto no V4.

8.6 A MELHORA, EM NÚMEROS

Como foi medido

Cada tarefa de documentação foi contada uma vez e classificada pela versão de prompt usada. Considerou-se **retrabalho** qualquer um destes sinais: pedir para refazer ou aprofundar a documentação do mesmo sistema, reenviar o mesmo prompt, pedir “continue”, ou corrigir idioma, pasta ou documento faltando.

Fase	Tipo de prompt	Tarefas	Com retrabalho	Taxa
V0	pipeline estruturado, PT	1	0	~0%
V1	pedidos livres, PT	12	6	~50%
V2	lista numerada, PT	3	2	~67%
V3	protocolo em inglês, sem regra de idioma	2	1	~50%
V4	prompt final, inglês + saída PT-BR	1	0	~0%
Prompts livres e lista (V1 + V2)		15	8	~53%
Prompts com contrato de execução (V0 + V3 + V4)		4	1	~25%



⚠️ Leia com cautela

É uma **estimativa indicativa**, não uma prova estatística:

- a amostra é pequena (19 tarefas no total);
- depois de 15/09 só houve uma documentação completa com o V4 registrada no histórico;
- parte das paradas veio de limite de uso da ferramenta, e não da qualidade do prompt;
- o V2 foi pior que o V1 provavelmente porque pedia muito mais (47 itens, PDFs, projeto PMI), o que levava ao limite de uso e a paradas no meio.

Os sinais qualitativos, porém, são claros: cada falha gerou uma regra, e as regras resolveram as falhas que as motivaram.

8.7 O PROMPT FINAL POR DENTRO

O prompt final (V4) está no arquivo `model/SYSTEM_DOCUMENTATION.md`, que acompanha este material. Ele tem 42 seções, em inglês. Os blocos mais importantes são estes.

8.7.1 Contrato de execução

O prompt começa definindo **papéis** para o agente: engenheiro full-stack sênior, DevOps, arquiteto de software, engenheiro de segurança, QA, analista de sistemas, redator técnico, engenheiro de banco de dados, SRE, gerente de projetos e analista de negócios. E também a postura de um **professor** que valoriza clareza, organização e construção gradual do entendimento.

A missão declarada **não** é “escrever documentação”. É descobrir, entender, fazer engenharia reversa, validar contra o código, identificar o comportamento real, documentar o estado atual, apontar defeitos e riscos, explicar para públicos técnicos e leigos, propor uma evolução justificada e, por fim, **validar de forma independente** o que foi produzido.

8.7.2 Idioma obrigatório

Tudo o que o agente produz (arquivos `.md` e `.pdf`, README, rótulos de diagramas, tabelas, manuais, planos e relatórios) deve estar em **português brasileiro** formal. As exceções são elementos técnicos que não se traduzem, como nomes de funções, variáveis e arquivos, comandos, bibliotecas e código-fonte. Mesmo nesses casos, a explicação em volta fica em português.

8.7.3 Nunca inventar

Todo fato sobre o sistema precisa de evidência: código, configuração, banco, migrações, testes, infraestrutura, documentação existente, logs. O que não puder ser confirmado recebe um rótulo explícito:

DESCONHECIDO · NÃO VERIFICADO · NÃO ENCONTRADO · INFERIDO
RECOMENDAÇÃO · PROPOSTA · REQUER VALIDAÇÃO

8.7.4 Separação de estados

A documentação nunca mistura:

AS-IS	como o sistema é hoje
TO-BE	como deveria ser depois de uma evolução proposta
GAP	a diferença entre os dois
RECOMMENDATION	uma melhoria tecnicamente justificada
PROPOSED IMPLEMENTATION	uma forma possível de implementar a recomendação

8.7.5 Somente leitura na descoberta

Durante a descoberta e a análise, o agente **não** refatora, não altera código, não apaga nem renomeia arquivos, não muda dependências, infraestrutura, banco ou configuração, e não executa operações destrutivas. Só pode criar documentação em docs/.

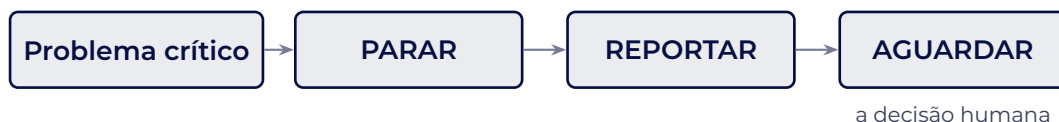
Descobrir antes de modificar

Isso reduz o risco de a IA fazer alterações prematuras baseadas numa compreensão incompleta do sistema.

8.7.6 Safe Stop-and-Report

Um agente que trabalha sobre um sistema real precisa saber **quando parar**. O protocolo lista as situações em que o agente deve interromper a operação e reportar, por exemplo:

- falta informação que o repositório não fornece;
- duas fontes de evidência se contradizem;
- seria preciso uma operação destrutiva ou afetar produção, banco real ou dados de usuários;
- o agente encontrou senhas, tokens, chaves ou dados pessoais;
- continuar exigiria inventar informação ou supor uma autorização que não foi dada;
- é preciso aprovação humana para seguir.



Cada parada gera um relatório padronizado (“SAFE STOP REPORT”) com o que estava sendo feito, o motivo, a evidência, os fatos verificados, o risco, o que *não* foi alterado e qual decisão é necessária. A gravidade vai de **STOP-P0** (crítico, parada imediata) a **STOP-P3** (baixo). E uma parada **não é fracasso**: é o comportamento correto diante do risco.

8.7.7 Documentação em fases

O prompt proíbe fazer tudo numa tacada só. O trabalho segue fases, cada uma com um artefato:

Fase	Objetivo	Artefato
0	Descoberta somente leitura: estrutura, tecnologias, dependências, APIs, banco	docs/00-discovery/DISCOVERY.md
1	Mapa do sistema: componentes, relações, entrada, processamento e saída de dados	docs/00-discovery/SYSTEM_MAP.md
2	Análise profunda: fluxos, regras, persistência, autenticação, integrações	(alimenta as seguintes)
3	Modelo de conhecimento: entidades, regras, fluxos, riscos, lacunas	docs/00-discovery/KNOWLEDGE_MODEL.md
4	Validação do entendimento: contradições, conclusões sem evidência, esquecimentos	docs/00-discovery/VALIDATION.md
5+	Documentação técnica, regras de negócio, dados, APIs, segurança, operação, README, guia para leigos, projeto de evolução e auditorias	docs/

Por que em fases

Para evitar a cascata “entendimento errado → documentação errada → recomendação errada → projeto futuro errado”. Em vez disso: **descobrir → evidenciar → modelar → validar → documentar → auditar → evoluir**.

8.7.8 O restante do prompt

As demais seções pedem: visão geral, arquitetura e tecnologias; regras de negócio (atuais e recomendadas); arquitetura de dados; APIs; telas; segurança; LGPD (RIPD/D-PIA); defeitos, falhas, fraquezas e dívida técnica; modelo de qualidade e testes; operação, Docker e recuperação de desastres; plano de manutenção e de desenvolvimento; modelo de ameaças; ADRs (registros de decisão de arquitetura); governança; glossário; diagramas; requisitos; README profissional; documentação para leigos (manual do usuário, guia completo, runbook); projeto de evolução alinhado ao PMI; priorização de P0 a P3; rastreabilidade; auditorias de qualidade e de completude; geração de PDF; e um relatório executivo final. O fecho resume o espírito:

Princípio final do prompt

“Your objective is not to make the system look better than it actually is. Your objective is to make the system **understood exactly as it is**.”

“O objetivo não é fazer o sistema parecer melhor do que é. O objetivo é fazer o sistema ser **entendido exatamente como ele é**.”

8.8 PROMPT EM INGLÊS, DOCUMENTAÇÃO EM PORTUGUÊS

O prompt final foi mantido em inglês. A motivação adotada foi a **economia de tokens**: os modelos de IA cobram e limitam o trabalho por *tokens* (pedaços de texto), e textos em inglês costumam ser divididos em menos tokens que o mesmo conteúdo em português.

⚠ O que os dados mostram

O histórico confirma a troca de idioma e o efeito colateral dela, mas **não mediu** a economia de tokens. Trate essa economia como uma prática adotada, não como resultado comprovado aqui.

O efeito colateral foi real: no dia 14/09, o prompt V3, só em inglês, fez o agente responder em inglês. No dia seguinte, o V4 ganhou a regra de idioma. A lição:

📖 Idioma do prompt × idioma do resultado

São coisas diferentes. Se você escreve o prompt em inglês e quer o resultado em português, **diga isso explicitamente**.

8.9 O QUE O PROMPT PEDE E O QUE ELE ENTREGA

O prompt pode ser resumido em seis grandes perguntas:

Descoberta	O que existe neste sistema?
Compreensão	Como as partes do sistema se relacionam?
Funcionamento	Como o sistema funciona?
Auditoria	Quais são os problemas, riscos e pontos fracos?
Documentação	Como transformar esse conhecimento em documentação útil?
Validação	Como verificar se a documentação corresponde ao sistema real?

O retorno esperado não é “vários arquivos Markdown”. É transformar “código que poucas pessoas conhecem” em “sistema cujo funcionamento pode ser compreendido e mantido por outras pessoas”. Uma pessoa nova deve conseguir, aos poucos: entender o propósito do sistema, instalar o ambiente, compreender a arquitetura e o banco, entender os principais fluxos, localizar funcionalidades, identificar dependências, diagnosticar problemas, fazer manutenção e continuar a evolução.

Um exemplo de estrutura produzida (a exata se adapta a cada sistema):

```
docs/
├── 00-discovery/
│   ├── DISCOVERY.md
│   ├── SYSTEM_MAP.md
│   ├── KNOWLEDGE_MODEL.md
│   └── VALIDATION.md
├── architecture.md
├── business-rules.md
├── database.md
├── api.md
├── security.md
├── known-issues.md
├── technical-debt.md
└── guia-para-leigos.md
```

8.10 COMO USAR NA PRÁTICA

⚠ Autorização primeiro

Antes de rodar um agente de IA sobre um sistema da instituição, confirme que o uso está autorizado e segue as normas de segurança da informação da sua OM. O agente lê o código e o envia a um serviço externo para ser processado.

1. Instale o Claude Code seguindo a documentação oficial: <https://docs.claude.com/en/docs/claude-code/overview>.
2. Garanta que o projeto está versionado e **limpo** (`git status` sem pendências). Assim, qualquer alteração do agente aparece no `git diff` e pode ser desfeita.
3. Crie uma branch para a documentação: `git switch -c docs/documentacao-inicial`.
4. Abra o terminal na pasta do projeto e inicie o agente com `claude`.
5. Entregue o prompt: cole o conteúdo ou peça para o agente ler o arquivo do prompt.
6. Acompanhe. Quando o agente parar e reportar (*safe stop*), leia o relatório e decida.
7. **Valide** a documentação contra o sistema real (veja a lista abaixo).
8. Registre a documentação com `commit`, envie a branch e abra um Merge Request, para que a equipe revise.

Lista de validação

- A arquitetura documentada corresponde ao código?
- Os endpoints citados realmente existem?
- O banco de dados corresponde ao que está escrito?

- As regras de negócio estão corretas? (pergunte a quem usa o sistema)
- As instruções de instalação funcionam numa máquina limpa?
- Os problemas apontados são reais?
- O que ficou marcado como DESCONHECIDO ou REQUER VALIDAÇÃO foi tratado?
- Nenhuma senha, token ou dado pessoal foi copiado para a documentação?

 Dica

A documentação é um **artefato vivo**. Quando o sistema mudar, ela também precisa ser revisada. Coloque a atualização da documentação como item do próprio Merge Request.

9

EXERCÍCIOS PRÁTICOS

Os exercícios seguem a ordem da apostila. As partes 1, 4 e 5 e o exercício 6.1 funcionam só com o Git instalado, sem internet e sem servidor. A parte 2 precisa de acesso a um GitLab. A parte 3 precisa de um agente de IA e de um projeto que você possa usar. Os exercícios 6.2 e 6.3 precisam de uma conta no GitHub ou no GitLab.

Dica

Faça os exercícios numa pasta de treino, fora de qualquer projeto real. Se algo der errado, apague a pasta e comece de novo: é para isso que ela existe.

9.1 PARTE 1 — GIT NO SEU COMPUTADOR

Exercício 1.1 — Primeiro repositório e primeiro commit

1. Crie um repositório local.
2. Crie um arquivo.
3. Execute `git add`.
4. Execute `git commit`.

>_ TERMINAL

```
mkdir exercicio-git
cd exercicio-git
git init -b main
echo "Olá, Git!" > saudacao.txt
git status
git add saudacao.txt
git commit -m "Cria a saudação"
```

Como saber se deu certo

`git log --oneline` mostra uma linha com “Cria a saudação”, e `git status` diz que não há nada a submeter.

🔨 Exercício 1.2 — Branch, alteração e merge

5. Crie uma branch.
6. Altere o arquivo.
7. Crie outro commit.
8. Faça o merge na main.

>_ TERMINAL

```
git switch -c feature/despedida
echo "Até logo!" >> saudacao.txt      # >> acrescenta uma linha no fim
git commit -am "Adiciona a despedida" # -a prepara os arquivos já acompanhados
git switch main
git merge feature/despedida
cat saudacao.txt
```

✅ Como saber se deu certo

O merge diz “Fast-forward” e o arquivo tem duas linhas: “Olá, Git!” e “Até logo!”.

🔨 Exercício 1.3 — Simular e resolver um conflito

9. Simule um conflito.
10. Resolva o conflito.

A ideia é mudar **a mesma linha** de formas diferentes em duas branches.

>_ TERMINAL

```
# na branch nova, a saudação fica formal
git switch -c feature/formal
printf "Prezados, bom dia!\nAté logo!\n" > saudacao.txt
git commit -am "Deixa a saudação formal"

# na main, a mesma linha fica informal
git switch main
printf "E aí, pessoal!\nAté logo!\n" > saudacao.txt
git commit -am "Deixa a saudação informal"

# agora junte as duas
git merge feature/formal
```

🖨 SAÍDA

```
Mesclagem automática de saudacao.txt
CONFLITO (conteúdo): conflito de mesclagem em saudacao.txt
```

Automatic merge failed; fix conflicts and then commit the result.

Abra `saudacao.txt` num editor de texto. Você vai ver os marcadores explicados na seção 2.6. Escolha a versão que fica (ou escreva uma terceira), apague os marcadores, salve e registre:

>_ TERMINAL

```
git add saudacao.txt
git commit -m "Resolve o conflito da saudação"
git log --oneline --graph --all
```

SAÍDA

```
* 757aae3 Resolve o conflito da saudação
|\
| * c2524a8 Deixa a saudação formal
* | a41aded Deixa a saudação informal
|/
* afe16bb Adiciona a despedida
* 4203776 Cria a saudação
```

✓ Como saber se deu certo

O desenho do `git log --graph` mostra as duas linhas se separando e se juntando de novo no commit de resolução. Os códigos dos commits no seu computador serão outros.

9.2 PARTE 2 – GITLAB

🛠 Exercício 2.1 — Um Merge Request completo

1. Crie ou use um projeto no GitLab.
2. Crie uma branch.
3. Faça uma alteração.
4. Faça push.
5. Crie um Merge Request.
6. Revise a alteração.
7. Faça o merge.

>_ TERMINAL

```
git clone <URL_DO_PROJETO_DE_TREINO>
cd <pasta-do-projeto>
git switch -c exercicio/meu-nome
echo "Treinamento de Git na Marinha" >> README.md
git add README.md
git commit -m "Registra o treinamento no README"
git push origin exercicio/meu-nome
```

Depois, no navegador:

1. abra o projeto e clique em **Create merge request** (ou vá em **Merge requests > New merge request**);
2. confira: origem exercicio/meu-nome, destino main;
3. escreva um título e uma descrição e crie o MR;
4. abra a aba **Changes** e deixe um comentário numa linha;
5. peça a um colega para aprovar e faça o merge (se o projeto permitir).

 Pelo terminal

Os passos no navegador podem ser trocados por `glab mr create -b main`, `glab mr view` e `glab mr merge` (capítulo 4).

 Como saber se deu certo

O MR aparece como “Merged”, e a linha nova está no `README.md` da main.

9.3 PARTE 3 – IA

 Escolha bem o projeto

Use um projeto pequeno, **pessoal ou de código aberto**. Não use código institucional sem autorização.

Exercício 3.1 — Descoberta com um agente de IA

Escolha um pequeno projeto existente e peça ao agente:

1. realizar descoberta somente leitura;
2. identificar tecnologias;
3. identificar a arquitetura;
4. identificar dependências;
5. identificar funcionalidades;

6. identificar problemas;
7. gerar documentação;
8. apresentar dúvidas e pontos que exigem validação humana.

Um ponto de partida é o prompt abaixo. Ele é uma **versão reduzida**, escrita para este exercício, que aproveita as travas do prompt final: somente leitura, nunca inventar, parar e perguntar.

>_ TERMINAL · PROMPT DE EXEMPLO

Você vai analisar este projeto. Siga estas regras sem exceção:

1. MODO SOMENTE LEITURA: não altere, não apague e não renomeie nenhum arquivo, e não faça commits. Você só pode criar arquivos em docs/.
2. NUNCA INVENTE: todo fato precisa de evidência (arquivo e linha). O que não puder confirmar, marque como DESCONHECIDO, INFERIDO ou REQUER VALIDAÇÃO.
3. PARE E PERGUNTE: se encontrar contradição, senha, token, dado pessoal ou algo que exija decisão humana, pare e me relate antes de seguir.
4. IDIOMA: escreva tudo em português brasileiro.

Tarefa:

- a) Crie docs/DISCOVERY.md com a estrutura de pastas, as tecnologias e as dependências encontradas.
- b) Crie docs/ARQUITETURA.md explicando os componentes e como se comunicam, com um diagrama em texto.
- c) Crie docs/FUNCIONALIDADES.md listando o que o sistema faz.
- d) Crie docs/PROBLEMAS.md com defeitos, riscos e dívida técnica.
- e) Termine com uma lista de perguntas que eu preciso validar.

🛠 Exercício 3.2 — Validar o que a IA produziu

Pegue três afirmações da documentação gerada e confira cada uma no código. Anote: estava certa, errada ou incompleta? Responda também às perguntas que o agente deixou no item (e).

✅ Como saber se deu certo

`git status` mostra **só** arquivos novos em docs/. Se aparecer qualquer outro arquivo modificado, o agente desrespeitou a regra 1: use `git diff` para ver o que mudou e `git restore` para desfazer.

9.4 PARTE 4 — DESAFIO: REPRODUZIR O PROBLEMA 1

🔨 Exercício 4.1 — O caso real, sem servidor

Simule o GitLab institucional com um repositório “vazio de trabalho” (*bare*) numa pasta do seu computador, com a *main* e o README padrão. Depois, integre a ele um sistema que já tem histórico próprio, seguindo os oito passos do capítulo 7.

Preparação: o “GitLab” de mentira.

>_ TERMINAL

```
mkdir desafio && cd desafio
git init --bare -b main gitlab.git      # faz o papel do servidor
git clone gitlab.git instituicao
cd instituicao
echo "# Projeto (README padrão da instituição)" > README.md
git add README.md
git commit -m "Initial commit"
git push origin main
cd ..
```

Preparação: o sistema que já existia.

>_ TERMINAL

```
mkdir sistema && cd sistema
git init -b main
echo "# Sistema de Controle" > README.md
echo "print('ola')" > app.py
git add README.md app.py
git commit -m "Primeira versão"
echo "x = 1" >> app.py
git commit -am "Ajuste"
```

Agora é com você. Ainda dentro de *sistema/*, aplique os oito passos. Como o “servidor” é uma pasta, a URL é o caminho dela: *../gitlab.git*. O passo 8 (Merge Request) não existe numa pasta, então pare no passo 7.

1. Antes do passo 4, tente `git merge origin/main` sem o parâmetro. Qual mensagem aparece? Por quê?
2. No passo 5, mantenha o README do **sistema**. Qual comando você usou: `--ours` ou `--theirs`?
3. Depois do passo 7, rode `git log --oneline --graph --all`. Onde está o commit que une os dois históricos?

 Respostas

1. fatal: refusing to merge unrelated histories. Os dois repositórios nasceram separados e não têm ancestral comum.
2. `git restore --ours README.md`: durante o merge você está na branch desenvolvimento, que tem o README do sistema.
3. No topo: o commit “Merge remote-tracking branch 'origin/main' into desenvolvimento”, com duas linhas chegando nele. Uma traz “Primeira versão” e “Ajuste”, a outra traz “Initial commit”.

9.5 PARTE 5 — PADRÕES DE COMMIT

Exercício 5.1 — Reescrever mensagens ruins

Reescreva cada mensagem abaixo no formato Conventional Commits (capítulo 5). Escolha o tipo, invente um escopo que faça sentido e escreva uma descrição curta.

1. “arrumei o erro que dava quando a senha tava vazia”
2. “coloquei explicação de como instalar no readme”
3. “mudei a indentação do arquivo de rotas”
4. “agora o relatório sai em pdf também”
5. “atualizei o pipeline pra rodar os testes”
6. “a API agora exige token em tudo, quem não mandar leva 401”

Exercício 5.2 — Aplicar no seu repositório de treino

No repositório `exercicio-git` da Parte 1, faça três commits pequenos, cada um com um tipo diferente (feat, docs e fix). Depois rode `git log --oneline` e confira se dá para entender o histórico só lendo as mensagens.

Respostas possíveis do 5.1

1. fix(login): aceita tentativa com senha vazia sem erro
2. docs(readme): explica a instalação
3. style(rotas): corrige a indentação
4. feat(relatorio): exporta em PDF
5. ci: roda os testes no pipeline
6. feat(api)!: exige token em todos os endpoints

No item 6, o ! marca a quebra de compatibilidade. Um rodapé **BREAKING CHANGE**: clientes sem token recebem 401 deixaria isso ainda mais claro. Os escopos e as palavras podem variar: o importante é o tipo certo e uma descrição que diga o que mudou.

9.6 PARTE 6 — ASSINATURA DE COMMITS

🛠 Exercício 6.1 — Ver o problema com os próprios olhos

Num repositório de treino, faça um commit com um nome e um e-mail que não são seus e confira o que o `git log` mostra:

>_ TERMINAL

```
git -c user.name="Pessoa Qualquer" -c user.email="qualquer@exemplo.com" \  
    commit --allow-empty -m "Quem fez este commit?"  
git log --format='%an <%ae> | assinatura: %G?' -1
```

🖨 SAÍDA

```
Pessoa Qualquer <qualquer@exemplo.com> | assinatura: N
```

O Git aceitou a identidade falsa, e o N diz que não há assinatura.

🛠 Exercício 6.2 — Assinar de verdade

Siga o capítulo 6 com o método que preferir (GPG ou SSH):

1. crie ou reaproveite uma chave;
2. cadastre a chave no GitHub ou no GitLab;
3. configure o Git para assinar todo commit;
4. faça um commit num repositório seu e envie com `git push`;
5. confira o selo na página de commits.

✅ Como saber se deu certo

No terminal, `git log --format='%h %G? %s' -1` mostra a letra G. No site, o commit tem o selo **Verified**. Se aparecer **Unverified**, confira primeiro o e-mail: `git config user.email` precisa ser um e-mail verificado na sua conta.

🛠 Exercício 6.3 — Uma tag assinada

Crie e confira uma tag assinada:

>_ TERMINAL

```
git tag -s v1.0 -m "Versão 1.0"  
git tag -v v1.0  
git push origin v1.0
```

✔ Como saber se deu certo

○ `git tag -v` mostra a mesma confirmação de assinatura do `git log --show-signature`, e a tag aparece como **Verified** na página de tags do GitHub.

10

PARA LEVAR DAQUI

10.1 OS DOIS PROBLEMAS, LADO A LADO

Os dois problemas parecem diferentes, mas se completam:



10.2 CONCEITOS FUNDAMENTAIS

Git	Sistema de controle de versão.
GitLab	Plataforma de colaboração e gerenciamento baseada em Git.
Branch	Linha de desenvolvimento.
Commit	Registro de uma alteração no histórico.
Merge	Integração de históricos ou alterações.
Merge Request	Solicitação para integrar uma branch em outra, normalmente com revisão.
GitLab CLI	Ferramenta para interagir com o GitLab pelo terminal (gLab).
Conflito	Situação em que o Git não consegue decidir sozinho como combinar alterações.
IA	Ferramenta auxiliar de análise, desenvolvimento e documentação, sob orientação e validação humana.

10.3 FLUXOS RECOMENDADOS

Para um novo desenvolvimento



Para um sistema legado

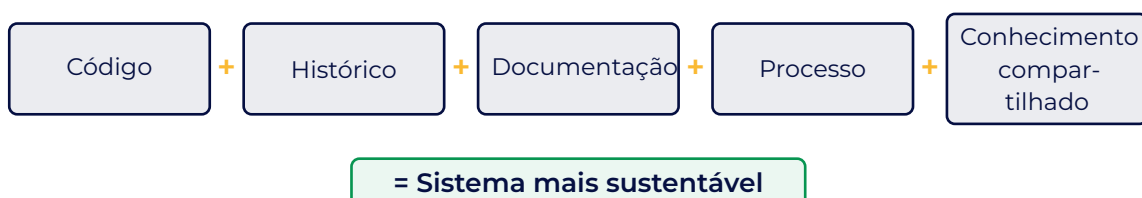


10.4 MENSAGEM FINAL

O conhecimento técnico de uma equipe não deve permanecer apenas na cabeça de quem desenvolveu o sistema.

- O código deve ter **histórico**.
- O sistema deve ter **documentação**.
- As alterações devem ter **revisão**.
- As decisões importantes devem ter **contexto**.

As ferramentas de IA podem acelerar esse trabalho, desde que usadas dentro de um processo controlado, com objetivos claros, restrições bem definidas e validação humana.



**TORNAR OS SISTEMAS MAIS FÁCEIS DE COMPREENDER,
MANTER, COMPARTILHAR E EVOLUIR.**



COLA DE COMANDOS

Configuração

<code>git config --global user.name</code> <code>"..."</code>	seu nome
<code>git config --global user.email</code> <code>"..."</code>	seu e-mail
<code>git config --list</code>	ver a configuração

Começar

<code>git init -b main</code>	criar repositório
<code>git clone <URL></code>	copiar um remoto

Registrar

<code>git status</code>	estado atual
<code>git diff</code>	o que mudou
<code>git add arquivo</code>	preparar
<code>git commit -m "..."</code>	registrar
<code>git commit -am "..."</code>	preparar e registrar os já acompanhados
<code>git log --oneline --graph --all</code>	histórico desenhado

Branches

<code>git branch</code>	listar
<code>git switch -c nome</code>	criar e entrar
<code>git switch nome</code>	trocar
<code>git branch -m antigo novo</code>	renomear
<code>git merge nome</code>	trazer nome para a atual
<code>git merge --abort</code>	cancelar o merge

Conflitos

<code>git restore --ours arq</code>	manter o lado da branch atual
<code>git restore --theirs arq</code>	manter o lado que chega
<code>git add arq</code>	marcar como resolvido

Remoto

<code>git remote -v</code>	listar remotos
<code>git remote add origin <URL></code>	adicionar remoto
<code>git fetch origin</code>	baixar sem mesclar
<code>git pull</code>	baixar e mesclar
<code>git push origin branch</code>	enviar

Desfazer

<code>git restore arq</code>	descartar mudança não preparada
<code>git restore --staged arq</code>	tirar da preparação
<code>git revert <commit></code>	desfazer com novo commit

Assinatura

<code>gpg --full-generate-key</code>	criar chave GPG
<code>gpg --list-secret-keys</code>	ver as chaves GPG
<code>--keyid-format=long</code>	
<code>gpg --armor --export ID</code>	exportar a chave pública
<code>git config --global gpg.format ssh</code>	assinar com SSH
<code>git config --global user.signingkey ...</code>	escolher a chave
<code>git config --global commit.gpgsign true</code>	assinar todo commit
<code>git commit -S -m "..."</code>	assinar um commit
<code>git log --show-signature -1</code>	conferir a assinatura
<code>git log --format='%h %G? %s'</code>	estado da assinatura (G, N, B, E)
<code>git tag -s v1.0 -m "..."</code>	criar tag assinada

GitLab CLI

<code>glab auth login</code>	autenticar
<code>glab mr create</code>	criar MR
<code>glab mr list</code>	listar MRs
<code>glab mr view</code>	ver o MR
<code>glab mr merge</code>	fazer o merge
<code>glab ci status</code>	ver o pipeline



GLOSSÁRIO

Agente de IA

Programa baseado em modelo de linguagem que executa tarefas num ambiente (ler arquivos, rodar comandos, escrever documentos), e não só conversa.

Ancestral comum

Commit a partir do qual duas branches se separaram. Sem ele, o Git recusa o merge comum.

Área de preparação (*staging*)

Onde ficam as mudanças escolhidas para o próximo commit.

Assinatura de commit

Marca criptográfica, feita com a sua chave privada, que prova quem criou o commit. Gera o selo *Verified*.

Bare repository

Repositório sem pasta de trabalho, só com o histórico. É o que um servidor guarda.

Branch

Linha de desenvolvimento; tecnicamente, um ponteiro para um commit.

Branch protegida

Branch que não aceita push direto; mudanças só por Merge Request.

CI/CD

Integração e entrega contínuas: testes e implantação automáticos a cada mudança.

Chave pública e chave privada

Par de chaves: a privada fica só com você e assina; a pública pode ser distribuída e serve para conferir a assinatura.

Commit

Registro de um estado do projeto, com autor, data e mensagem.

Conventional Commits

Especificação de mensagens de commit no formato `tipo(escopo): descrição`.

Conflito

Quando duas mudanças na mesma região de um arquivo não podem ser combinadas automaticamente.

Fast-forward

Merge em que o Git só avança o ponteiro, porque não houve trabalho paralelo.

Gitmoji

Convenção que associa um emoji a cada tipo de mudança, escrito como código (`:sparkles:`).

GPG

GnuPG, programa que cria chaves e assina dados. Uma das duas formas de assinar commits.

GAP

Lacuna entre o estado atual (AS-IS) e o desejado (TO-BE), ou informação que falta e não deve ser inventada.

HEAD

Referência para onde você está agora (em geral, a branch atual).

Merge

Integração de duas linhas de histórico.

Merge Request (MR)

Pedido para integrar uma branch em outra, com revisão. No GitHub: *Pull Request*.

origin

Nome padrão do repositório remoto principal.

ours / theirs

Num merge: o lado da branch atual / o lado que está chegando.

Pipeline

Sequência automática de etapas (compilar, testar, publicar) executada pelo GitLab.

Prompt

Instrução dada a um modelo ou agente de IA.

Remoto

Cópia do repositório em outro lugar, como o GitLab.

Repositório

Pasta cujo histórico é controlado pelo Git.

Self-Managed

GitLab instalado e administrado pela própria organização.

Token

Na IA, pedaço de texto usado para medir custo e limite. No GitLab, credencial de acesso (trate como senha).

Verified / Unverified

Selos do GitHub e do GitLab: assinatura conferida, ou assinatura presente mas que não pôde ser conferida.



PLANO DA APRESENTAÇÃO

Roteiro de tempo da capacitação de 01/10/2026, das 10h às 11h. Os números de slide se referem a apresentacao/apresentacao.pdf.

Horário	Duração	Slides	Conteúdo
10h00	3 min	1–2	Abertura, agenda e objetivos
10h03	19 min	3–15	Fundamentos: por que versionar, Git, GitLab, Self-Managed, branch, Merge Request, comandos, fluxo, conflito, glab
10h22	15 min	16–25	Problema 1: cenário, históricos sem relação, os oito passos, ours e theirs
10h37	18 min	26–35	Problema 2: IA como ferramenta, evolução dos prompts, falhas e regras, números, fases, travas, idioma, entregas
10h55	5 min	36–39	Encerramento, mensagem final e perguntas

Dica

Se o tempo apertar, os slides 8 (recursos do GitLab), 12 (tabela de comandos) e 35 (o que o prompt entrega) podem ser passados rapidamente: o conteúdo deles está completo nesta apostila.



CRÉDITOS E LICENÇAS

Todo o material de terceiros usado aqui está salvo localmente, na pasta `assets/` (imagens e fontes), e o manual da marca em `materials/manual-da-marca/`, para que a apostila e a apresentação possam ser recompiladas sem internet.

Item	Origem	Licença
Manual de Identidade Visual da Marinha do Brasil (cores e tipografia)	CCEM, dez/2025 (<code>materials/manual-da-marca/</code>)	Material institucional da Marinha do Brasil
Foto aérea da Ilha das Cobras (Arsenal de Marinha do Rio de Janeiro, 2021)	Marinha do Brasil, via Wikimedia Commons ("Arsenal de Marinha do Rio de Janeiro (AMRJ) (52652566017).jpg")	CC BY-SA 2.0
Logotipo do Git	Jason Long, via Wikimedia Commons	CC BY 3.0
Logotipo do GitLab (tanuki)	GitLab Inc., <code>gitlab-com/gitlab-artwork</code>	CC BY-NC-SA 4.0; marca registrada da GitLab Inc.
Símbolo do GitHub	GitHub Octicons, via Wikimedia Commons	MIT; marca da GitHub Inc.
Símbolo do Claude	Wikimedia Commons	CC0; marca da Anthropic
Tirinha xkcd #1597 "Git"	Randall Munroe, https://xkcd.com/1597/	CC BY-NC 2.5
Fonte Montserrat	Julieta Ulanovsky e colaboradores	SIL Open Font License 1.1
Fonte Bebas Neue	Dharma Type, via Google Fonts	SIL Open Font License 1.1
Fonte Fira Mono	Mozilla / Carrois	SIL Open Font License 1.1

As marcas de terceiros (Git, GitLab, GitHub, Claude) aparecem apenas para identificar as ferramentas de que o texto trata. As licenças NC (não comercial) são compatíveis com o uso deste material: capacitação interna, sem fins lucrativos.

Identidade visual

As cores seguem o Manual de Identidade Visual da Marinha do Brasil: Azul Marinha #050F41, Amarelo Ouro #FAB932 e Verde Brasil #079551. A tipografia de apoio oficial é a Montserrat, usada

no texto. As tipografias de título do manual (Octin College e Bebas Neue Pro) são comerciais. No lugar delas, usamos a Bebas Neue livre, do mesmo desenho da alternativa oficial.



PARA CONTINUAR ESTUDANDO

- **Pro Git**, de Scott Chacon e Ben Straub. O livro de referência do Git, gratuito e com tradução para o português: <https://git-scm.com/book/pt-br/v2>.
- **Documentação oficial do Git**: <https://git-scm.com/docs>.
- **Documentação do GitLab**: <https://docs.gitlab.com>. Veja em especial *Merge requests* e *Protected branches*.
- **GitLab CLI (glab)**: <https://gitlab.com/gitlab-org/cli>.
- **Claude Code**: <https://docs.claude.com/en/docs/claude-code/overview>.
- **Conventional Commits**: <https://www.conventionalcommits.org/pt-br/v1.0.0/>.
- **Gitmoji**: <https://gitmoji.dev>.
- **Padrões de commits**, de Iuri Silva, uma referência em português: <https://github.com/iuricode/padroes-de-commits>.
- **Assinatura de commits no GitHub**: <https://docs.github.com/pt/authentication/managing-commit-signing>.
- **Assinatura de commits no GitLab**: https://docs.gitlab.com/user/project/repository/signed_commits/.
- **Learn Git Branching**, exercícios visuais e interativos de branch e merge, em português: https://learngitbranching.js.org/?locale=pt_BR.

Arquivos deste material

<code>apresentacao/apresentacao.pdf</code>	os slides da capacitação
<code>apostila/apostila.pdf</code>	esta apostila
<code>model/SYSTEM_DOCUMENTATION.md</code>	o prompt final (V4) de documentação
<code>roteiro_versao_01.md</code>	o roteiro inicial, com os assuntos obrigatórios
<code>roteiro_versao_final.md</code>	o roteiro final
